CSA47 - DEEP LEARNING

CO1 ASSESSMENT 1

Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

AgriVision - Experimental Report

Field	Details
Student name	Thikkavarapu Sanhith
Register number	192311228
Course	CSA4704 - Deep Learning
Assessment	CO1 Assessment 1 - Individual assignment
Total marks	50
Dataset	PlantVillage color images; tomato subset for Tasks 1-11; 38 classes for Task 12
Environment	Kaggle Notebooks / Python

Report structure

- Executive summary and experimental setup
- Task 1 - Problem structure and data representation
- Task 2 - Linear algebra, PCA and SVD
- Task 3 - Probability and information theory
- Task 4 - Numerical computation issues
- Task 5 - Baseline learning strategies
- Task 6 - Capacity, overfitting and underfitting
- Task 7 - Bias-variance and Bayesian estimation
- Task 8 - Gradient-based optimisation
- Task 9 - Experimental performance
- Task 10 - Theory versus experiments
- Task 11 - Whether deep learning is justified
- Task 12 - Changed requirements design challenge
- Limitations and final conclusion
- Appendix A - Captured Kaggle output records
- Appendix B - Consolidated reproducibility code

Conventions: final calculated values are shown to four decimal places. Image counts and tensor dimensions are shown as integers. Reported timings are measured in the Kaggle environment and can vary on rerun.

Executive summary

This assessment studies whether classical machine learning is adequate for a crop-leaf disease screening task. The experimental setup follows the official brief: public leaf-disease images, a 64 x 64 grayscale representation (4,096 features), training-set scales of 1,000, 5,000, 10,000 and the full available training split, and a final redesign analysis for 38 classes, 20% noisy labels, a 50 ms latency target and a 5 MB memory target.

The selected PlantVillage color folders contain 54,305 images in 38 classes. The tomato subset contains 18,160 images across 10 tomato classes. The original tomato experiment uses 12,712 training images, 2,724 validation images and 2,724 test images. The 38-class challenge uses 38,013 training images, 8,146 validation images and 8,146 test images.

The strongest tested classical baseline is k-nearest neighbours with 100 PCA components: validation accuracy 0.5683 and macro-F1 0.4936. The capacity study finds depth 8 (C3) has the lowest validation error among the five tree depths tried, while depths 16 and 32 show a large train-validation gap. The 38-class experiment gives the selected noisy-label SGD model test accuracy 0.3451 and test macro-F1 0.2266. Its saved pipeline is 1.5522 decimal MB and has a measured Kaggle p95 latency proxy of 0.3858 ms on an already resized 64 x 64 image.

Decision: Redesign before deployment. Resource proxies are under the specified limits for the selected SGD configuration, but predictive performance is too weak and the SGD training run reported a convergence warning. Smartphone latency and peak memory have not been measured on the target device. A CNN was not trained in the recorded experiments, so deep learning is justified for further testing but has not been proven superior by a direct comparison.

Experimental setup and reproducibility

Item	Choice used in this report
Dataset source	PlantVillage public dataset, color representation only
Original experiment	All 10 tomato classes because the three-class example has only 4,500 images and cannot supply a 5,000 or 10,000 image training subset
Image preprocessing	Convert to grayscale; resize to 64 x 64 with Lanczos resampling; flatten to 4,096 values; normalise by dividing by 255 where required
Data splits	Stratified 70% training / 15% validation / 15% test; this was an experimental choice, not a percentage dictated by the assessment
Random seeds	42 for the principal train/validation/test split and most sampling steps
Feature compression	PCA fitted on the training split only; 100 components used for the primary baseline comparison
Evaluation	Validation data for model selection; held-out test used for the selected 38-class model
Rounding	Final calculations to 4 decimal places

Data-source counts and all measured results in this report are transcribed from the Kaggle outputs supplied during the work. Several runtime measurements vary slightly between cells because the models were evaluated in separate runs. Timing differences are reported rather than averaged together.

Task 1 - Analyse the problem structure and data representation (4 marks)

1.1 Learning-task decomposition

Component	AgriVision specification	Constraint / implication
Learning task	Supervised multi-class image classification	Training data must include images and disease labels
Input	A smartphone crop-leaf photograph; for the experiment, a 64 x 64 grayscale image	Resizing and grayscale conversion simplify inputs but discard fine detail and color cues
Output	One of 10 tomato classes for Tasks 1-11; 38 crop-disease classes in Task 12	More classes increase the output label space and make some confusions more likely
Performance measures	Accuracy, precision, recall, macro-F1, confusion matrix; runtime and size	Class imbalance means one accuracy score is not sufficient
Deployment objective	For Task 12: <= 50 ms per image and <= 5 MB model/memory limit	Predictive performance must be considered together with speed and storage

1.2 Scalars, vectors, matrices and tensors

Object	Shape	Interpretation
Single pixel intensity	Scalar	One grayscale pixel value from 0 to 255; after normalisation it lies in [0, 1]
One grayscale image	64 x 64 matrix	Two-dimensional arrangement retains explicit pixel neighbourhood
One flattened image	(4096,) vector	One feature vector; flattening keeps values but removes explicit 2-D structure
Mini-batch of 32 flattened images	(32, 4096) matrix	Rows are images and columns are pixel features
Mini-batch before flattening	(32, 64, 64) tensor	Batch axis plus row and column axes; a color batch would have an extra channel axis
Training inputs	(12712, 4096) matrix	Rows are training images and columns are features

Object	Shape	Interpretation
Labels	(12712,) vector	Each integer 0-9 maps to one tomato class; 38-class labels use IDs 0-37

1.3 L1, L2 and L-infinity norms

For the first training image, the code converts the 4,096 uint8 pixels to float64 and divides each value by 255. Let x_i denote the resulting normalised feature value.

L1 norm: ||x||_1 = sum_{i=1}^{4096} |x_i|

Substitution from the computed vector: |x_1| + |x_2| + ... + |x_4096| = 1631.4549. Final answer: ||x||_1 = 1631.4549.

Meaning for AgriVision: L1 is the total absolute pixel intensity. It describes the overall intensity mass but is not, by itself, a disease severity score.

L2 norm: ||x||_2 = sqrt(sum_{i=1}^{4096} x_i^2)

The computed sum of squares is 763.2129. Therefore, ||x||_2 = sqrt(763.2129) = 27.6263.

Meaning for AgriVision: L2 measures the overall magnitude of the image vector and is useful for comparing feature-vector magnitudes.

L-infinity norm: ||x||_infinity = max_i |x_i|

The largest normalised pixel value is 0.6824, so ||x||_infinity = 0.6824.

Meaning for AgriVision: L-infinity records the largest individual pixel intensity, not the full image intensity.

Verification: NumPy independently returned 1631.4549, 27.6263 and 0.6824, respectively. The assertions in the Kaggle cell passed.

1.4 Preprocessing effects and label imbalance

Step	Effect on representation	Risk / reason for use
Resize to 64 x 64	Each image has the same number of pixels	Smaller files and fixed model input; tiny spots or edges may be lost
Convert to grayscale	3 color channels become one intensity channel	Reduces features to 4,096, but removes color information useful for some diseases
Normalise /255	Pixel range changes from 0-255 to 0-1	Improves scale consistency for gradient-based learning
Flatten	64 x 64 matrix becomes a 4,096-vector	Classical models can consume a vector, but explicit spatial neighbourhoods are no longer represented

Step	Effect on representation	Risk / reason for use
Train/validation/test split	Creates disjoint data partitions	Validation supports model selection; the test set must remain untouched until final evaluation

The training class shares include 29.4997% Tomato Yellow Leaf Curl Virus and 2.0532% Tomato Mosaic Virus. The largest-to-smallest class-share ratio is 29.4997 / 2.0532 = 14.3677. This imbalance can bias overall accuracy toward common classes, so per-class recall and macro-F1 are needed.

Task 1 conclusion: representation choices reduce dimensionality and standardise inputs, but each can discard information. The model comparison must measure whether those changes improve classification, not assume they do.

Task 2 - Examine the feature space using linear algebra (5 marks)

2.1 SVD, eigenvalues and explained variance

$$X_c = U \Sigma V^T; \quad \lambda_i = \sigma_i^2 / (n - 1)$$

X_c is the centred training feature matrix; sigma_i is a singular value; lambda_i is the corresponding eigenvalue of the sample covariance matrix; n = 12712 training images. The PCA implementation used a randomized SVD solver with 256 components.

PC	Singular value	Covariance eigenvalue	Variance ratio	Cumulative variance
1	469.4708	17.3395	0.1966	0.1966
2	348.9064	9.5772	0.1086	0.3051
3	332.0560	8.6745	0.0983	0.4035
4	320.8000	8.0963	0.0918	0.4952
5	170.8500	2.2964	0.0260	0.5213
6	153.5338	1.8545	0.0210	0.5423
7	135.9263	1.4535	0.0165	0.5588
8	134.9085	1.4319	0.0162	0.5750
9	131.1254	1.3527	0.0153	0.5903
10	106.6960	0.8956	0.0102	0.6005

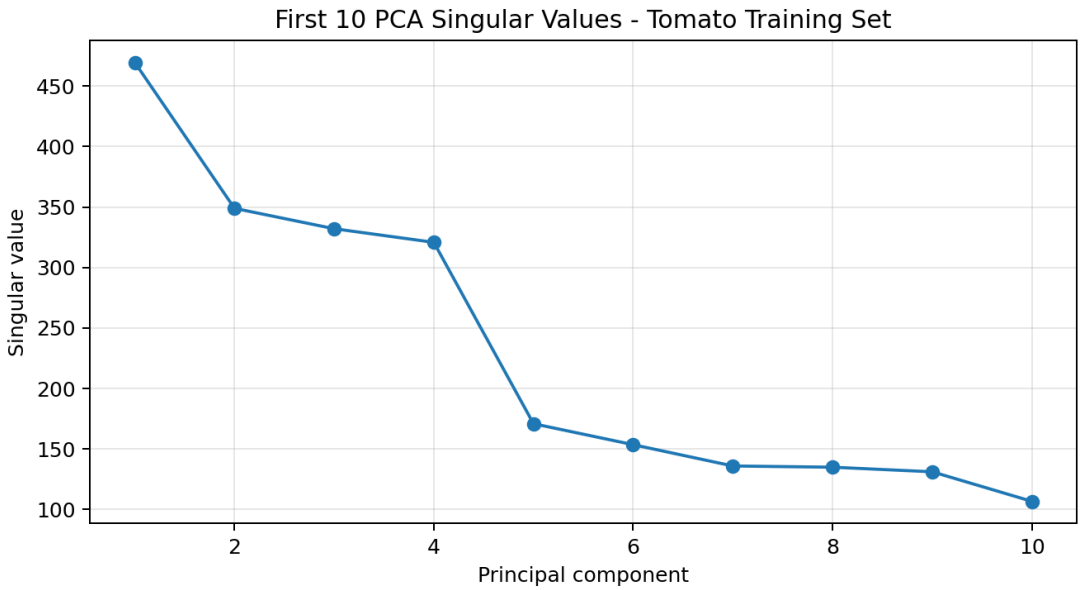


Figure 1. The first ten singular values from the tomato PCA run.

The first principal component accounts for 19.6600% of the variance, and the first ten account for 60.0481%. The first 256 components account for 89.0371%. The sharp fall in the spectrum indicates that variance is not evenly spread across all 4,096 pixel features; some directions contain much more variation than others.

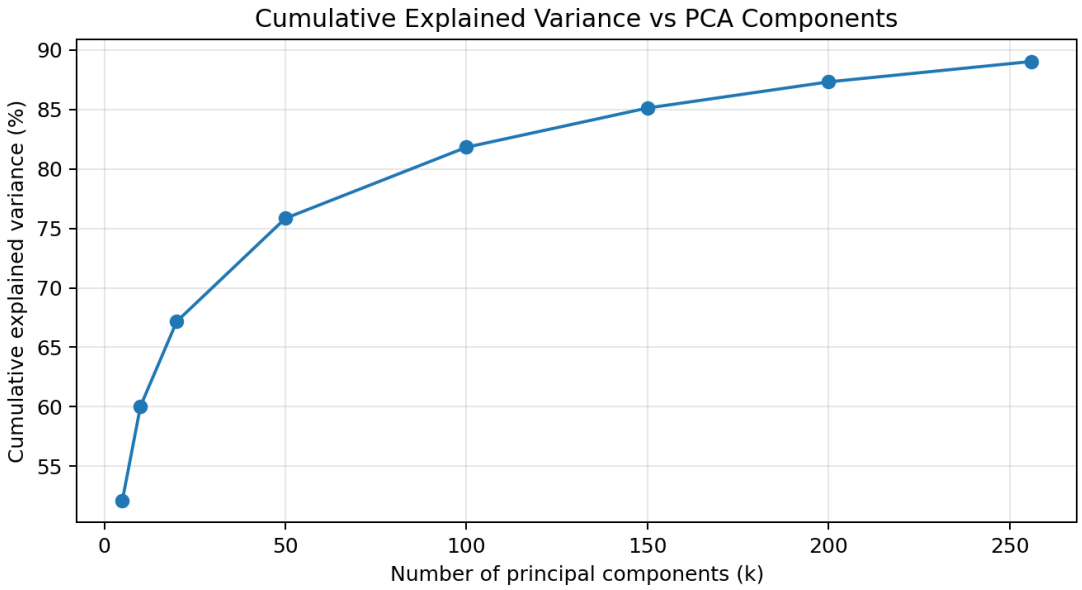


Figure 2. Cumulative explained variance for selected component counts.

2.2 Reconstruction-error trade-off

Relative reconstruction error E_k = ||X_c - Xhat_c^(k)||_F / ||X_c||_F

Components k	Explained variance (%)	Validation relative reconstruction error
5	52.1264	0.7058

Components k	Explained variance (%)	Validation relative reconstruction error
10	60.0481	0.6483
20	67.1677	0.5885
50	75.8431	0.5063
100	81.8237	0.4433
150	85.1243	0.4036
200	87.3225	0.3748
256	89.0371	0.3510

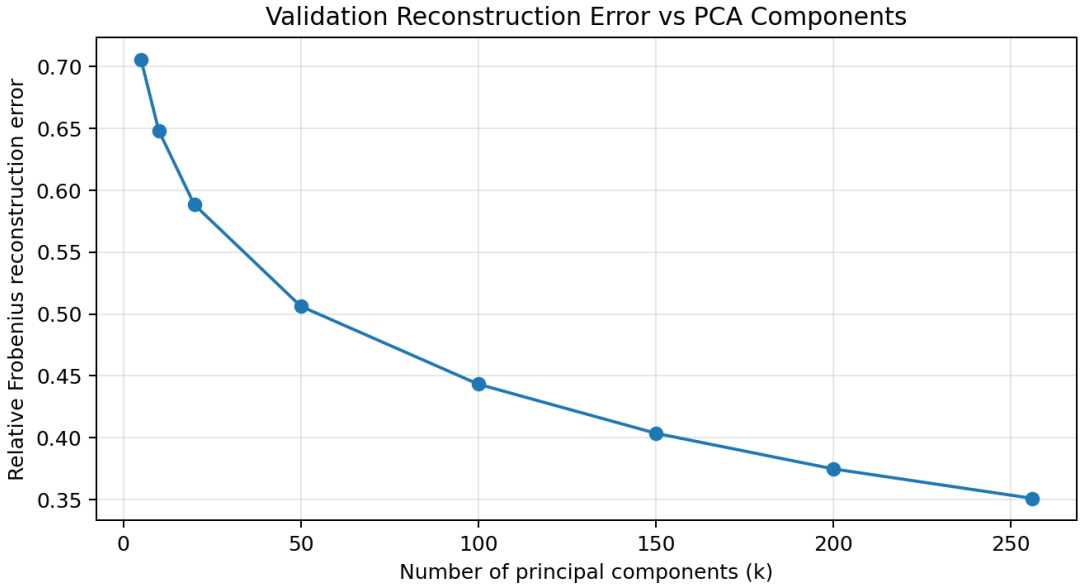


Figure 3. Reconstruction error decreases as more PCA components are retained.

At k = 100, 81.8237% of training variance is retained and the sampled validation reconstruction error is 0.4433. At k = 256, retained variance increases to 89.0371% and the error falls to 0.3510. There is no single best k on reconstruction error alone; Task 9 shows classification accuracy and resource costs also matter.

The 256-component PCA fit on the full 12,712-image tomato training matrix took 18.7822 s. PCA fitting time for 64 components on varying sizes was 2.7361 s (1,000 training images), 4.4388 s (5,000), 4.4004 s (10,000), and 5.7493 s (12,712). The general direction is an increase with dataset size, but timings are not strictly monotone because of runtime variability and algorithm overhead.

2.3 Hand-worked 2 x 2 eigendecomposition

The code extracted a covariance submatrix for pixel indices 2058 and 1994:

```
C = [[0.02932759, 0.02667736], [0.02667736, 0.02894818]]
```

For C = [[a,b],[b,d]], the characteristic equation is det(C - lambda I) = 0. Expanding gives lambda^2 - (a+d)lambda + (ad-b^2) = 0. Equivalently, the eigenvalues are:

```
lambda_(1,2) = ((a+d) +/- sqrt((a-d)^2 + 4b^2)) / 2
```

Substitution: a+d = 0.05827577; sqrt((a-d)^2 + 4b^2) approximately 0.0533560. Thus lambda_1 = 0.0558 and lambda_2 = 0.0025 to four decimal places. NumPy returned [0.0558, 0.0025].

The eigenvector matrix, with columns ordered by descending eigenvalue, was approximately Q = [[-0.7096, 0.7046], [-0.7046, -0.7096]]. The diagonal eigenvalue matrix was Lambda = [[0.0558, 0], [0, 0.0025]]. The reconstruction Q Lambda Q^T matched C, and the Frobenius reconstruction difference printed as 0.0000000000.

Meaning for AgriVision: this small subproblem demonstrates a dominant direction in the covariance of two selected pixels. It illustrates the mathematics of PCA; it is not enough by itself to justify the compression level for all 4,096 features.

> **Visual-component limitation:** The Kaggle cell generated the leading-component image figure as pca_component_images.png. That rendered figure was not uploaded into this chat, so this report does not assign a specific visual meaning to individual PCs based only on numerical eigenvalues. Inspect the saved figure in Kaggle and add a visual interpretation before final submission.

Task 3 - Analyse uncertainty using probability and information theory (4 marks)

3.1 Class entropy

```
H(p) = - sum_i p_i log2(p_i)
```

The validation class distribution across the ten tomato classes has measured entropy H(p) = 3.0298 bits. The maximum entropy for ten classes is log2(10) = 3.3219 bits. Therefore the normalised entropy is 3.0298 / 3.3219 = 0.9120.

Meaning for AgriVision: there is substantial class uncertainty, but the classes are not perfectly balanced. The training distribution has a ratio of 14.3677 between the largest and smallest class shares, so class-specific metrics are important.

3.2 Cross-entropy and KL divergence

```
H(p,q) = - sum_i p_i log2(q_i)

D_KL(p || q) = sum_i p_i log2(p_i / q_i)
```

Model	Features	Marginal cross-entropy (bits)	KL(p‖q) (bits)	KL(q‖p) (bits)	Per-image CE (bits)
SGD log loss	Raw 4096	3.2309	0.2011	0.1989	5.2895
k-NN k=5	Raw 4096	3.2528	0.2230	0.1952	13.8795
Decision tree	Raw 4096	3.0309	0.0010	0.0010	20.7338
SGD log loss	PCA-100	3.0409	0.0111	0.0110	2.3877
k-NN k=5	PCA-100	3.1093	0.0795	0.0702	8.8623

Model	Features	Marginal cross-entropy (bits)	KL(p‖q) (bits)	KL(q‖p) (bits)	Per-image CE (bits)
Decision tree	PCA-100	3.0319	0.0021	0.0020	20.8248

In this experiment p is the empirical marginal validation class distribution and q is the mean predicted probability distribution across validation images. Thus these marginal quantities are different from per-image cross-entropy, which evaluates probability assigned to each individual true label.

3.3 Verify H(p,q) = H(p) + D_KL(p||q)

```
H(p,q) = H(p) + D_KL(p || q)
```

For raw-feature SGD: 3.0298 + 0.2011 = 3.2309 bits. For k-NN with PCA-100: 3.0298 + 0.0795 = 3.1093 bits. The identity-check gap reported by Python was 0.0000 for every configuration.

Meaning for AgriVision: the cross-entropy identity shows that, for a fixed true distribution p, minimising cross-entropy is equivalent to minimising D_KL(p||q), because H(p) does not depend on model predictions q.

3.4 KL asymmetry and interpretation

For k-NN with PCA-100, D_KL(p||q) = 0.0795 bits while D_KL(q||p) = 0.0702 bits. The values differ because each direction weights the logarithmic ratio differently. KL divergence is non-negative and is zero only when the compared distributions match on all classes with positive probability.

The raw decision tree has marginal KL divergence 0.0010 bits but validation accuracy 0.4170, while its per-image cross-entropy is 20.7338 bits. Its average predicted class frequencies look similar to the true marginal distribution, yet its individual predictions can be poorly calibrated or confidently wrong. Marginal distribution agreement must not be mistaken for classification quality.

Task 4 - Diagnose numerical computation issues (3 marks)

4.1 Naive softmax failure

```
softmax(z_i) = exp(z_i) / sum_j exp(z_j)
```

For z = [1000, 999, 995], the direct exponentials overflowed: NumPy returned [inf, inf, inf]. Dividing by the infinite sum produced [nan, nan, nan].

4.2 Stable softmax

```
m = max(z);   softmax(z_i) = exp(z_i - m) / sum_j exp(z_j - m)
```

m = 1000 and z-m = [0, -1, -5]. The exponentials are [1.0000, 0.3679, 0.0067], with denominator 1.3746. The resulting probabilities are [0.7275, 0.2676, 0.0049] and their sum is 1.0000.

Subtracting the same m from every logit leaves the softmax unchanged because the common factor exp(-m) cancels in the numerator and denominator. It prevents overflow for this example.

4.3 Log-sum-exp and covariance conditioning

```
LSE(z) = m + ln(sum_j exp(z_j - m))
```

Using the stable exponentials, LSE(z) = 1000 + ln(1.3746) = 1000.3182.

Condition number: kappa_2(C) = lambda_max / lambda_min

For the retained 256-component PCA covariance subspace, lambda_max = 17.33952904 and lambda_min = 0.02197102. Therefore kappa_2(C) = 17.33952904 / 0.02197102 = 789.2001.

Meaning for AgriVision: a large condition number means directions have different scales/curvatures and can slow or destabilise gradient-based optimisation. Task 8 uses standardised PCA coordinates, so the condition number above is not a direct measurement of the standardised feature matrix used in that separate experiment.

Task 5 - Differentiate baseline learning strategies (4 marks)

Three distinct baseline algorithms were implemented: SGDClassifier with log loss (linear parametric model), k-nearest neighbours with k=5 (non-parametric, local-neighbour model), and a decision tree (tree-based partitioning model). Each was evaluated with raw 4,096 features and PCA-100 features.

Model	Representation	Accuracy	Macro-F1	Fit time (s)	Validation prediction time (s)
SGD log loss	Raw 4096	0.3990	0.3549	213.6941	0.0255
k-NN k=5	Raw 4096	0.4486	0.3644	0.0376	12.7039
Decision tree	Raw 4096	0.4170	0.3327	78.7767	0.0119
SGD log loss	PCA-100	0.4758	0.3456	3.7198	0.0082
k-NN k=5	PCA-100	0.5683	0.4936	0.0030	0.4062
Decision tree	PCA-100	0.4244	0.3523	4.2239	0.0036

The raw-feature SGD run reached the maximum number of iterations and emitted a convergence warning. The metrics should therefore be interpreted as results for the specified training settings, not as proof that this implementation reached its best linear-model solution.

The linear model has a fixed coefficient matrix and a relatively small stored model. k-NN stores the training examples and uses distances at inference time; this is expensive in high-dimensional space. The decision tree learns a set of hierarchical feature splits and has a compact inference path, but its accuracy and Macro-F1 are lower than PCA-100 k-NN in this experiment.

PCA-100 improved accuracy for all three models. Macro-F1 improved for k-NN and the tree but fell slightly for SGD from 0.3549 to 0.3456, reinforcing that compression can affect classes differently.

Task 6 - Analyse capacity, overfitting and underfitting (4 marks)

The extra instructions specify a capacity plot for C1-C5, including training and validation error, underfitting, overfitting, optimal capacity and the generalisation gap. This plot is included here under the closest matching official requirement, Task 6.

Error rate E = 1 - accuracy; generalisation gap G = E_validation - E_training

Capacity	Tree depth	Training error	Validation error	Gap
C1	2	0.6573	0.6553	-0.0020
C2	4	0.5753	0.5815	0.0062
C3	8	0.4541	0.5209	0.0668
C4	16	0.1340	0.5734	0.4394
C5	32	0.0944	0.5712	0.4768

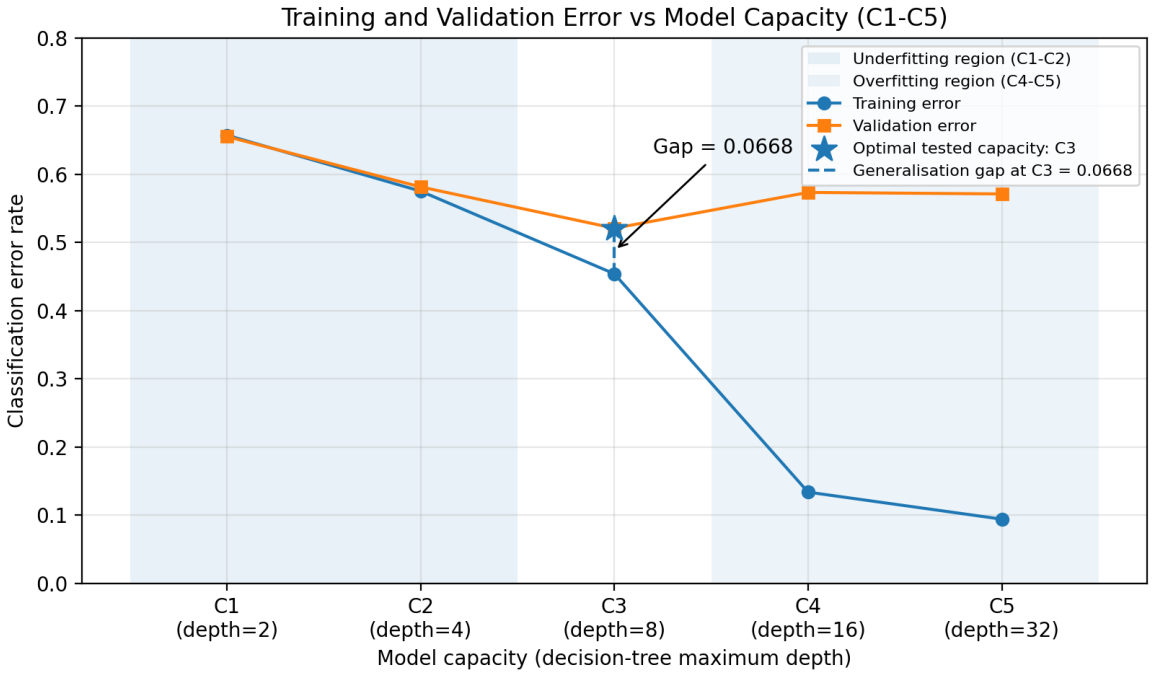


Figure 4. Required capacity curve: C1-C5, underfitting/overfitting regions, optimal tested capacity and generalisation gap.

C1 and C2 have high training and validation errors, which is consistent with underfitting. At C4 and C5, training error drops substantially while validation error remains high; the gaps are 0.4394 and 0.4768. C3 (depth 8) has the lowest validation error of the tested settings, 0.5209, so it is the optimal tested capacity, not a guaranteed global optimum.

Meaning for AgriVision: increasing tree depth can fit the training images more closely without improving unseen-image performance. The large gap at C5 indicates poor generalisation. Model selection should rely on validation data or cross-validation, not the test set, because repeated selection on the test set biases the reported test error.

Task 7 - Decompose error into bias and variance (4 marks)

7.1 Bootstrap experiment

Empirical multiclass Brier loss = mean_i sum_c (p_ic - y_ic)^2

Bias-squared proxy = mean_i sum_c (mean_b[p_bic] - y_ic)^2

Variance = mean_i sum_c Var_b(p_bic)

Model	Replicas	Squared-bias proxy	Variance	Mean Brier loss	Residual
Low capacity, depth 2	10	0.7851	0.0027	0.7878	0.0000
High capacity, depth 32	10	0.6166	0.5137	1.1303	0.0000

For depth 2: 0.7851 + 0.0027 = 0.7878. For depth 32: 0.6166 + 0.5137 = 1.1303. The residual reported by Python is approximately zero because the decomposition is defined from the same replicated probabilities and observed one-hot labels.

Interpretation: the deeper tree lowers the squared-bias proxy but has much greater prediction variance across bootstrap samples. The higher-capacity model is more sensitive to the data sample and has the larger mean Brier loss in this experiment.

Limitation: The empirical residual of zero does not mean irreducible label noise is zero. With one observed label per validation image, true conditional label noise cannot be isolated independently. This is stated explicitly because the assessment asks about bias-squared + variance + noise.

7.2 MLE versus MAP using Beta(2,2) prior

MLE: p_hat = s/n; MAP for Beta(alpha,beta): p_hat = (s + alpha - 1)/(n + alpha + beta - 2)

For the selected class Tomato Yellow Leaf Curl Virus, the chosen prior is Beta(2,2), so MAP = (s+1)/(n+2). At n=100, s=28: MLE = 28/100 = 0.2800; MAP = 29/102 = 0.2843.

n	Class count s	MLE	MAP Beta(2,2)	Absolute difference
100	28	0.2800	0.2843	0.0043
500	155	0.3100	0.3108	0.0008
1000	279	0.2790	0.2794	0.0004
5000	1451	0.2902	0.2903	0.0001
12712	3750	0.2950	0.2950	0.0000

Meaning for AgriVision: the Beta prior has a greater relative effect for small samples. As n grows, the MAP estimate approaches the MLE, and their difference rounds to zero at the full sample size.

Task 8 - Analyse gradient-based optimisation (4 marks)

8.1 Softmax-regression gradient derivation

Z = XW + b; p_ic = exp(z_ic) / sum_j exp(z_ij)

J(W,b) = -(1/N) sum_i sum_c y_ic ln(p_ic) + (lambda/2)||W||_F^2

For softmax with cross-entropy, the derivative of loss with respect to a logit is p_ic - y_ic. Applying the chain rule to Z = XW + b gives:

dJ/dW = X^T(P - Y)/N + lambda W

dJ/db = (1/N) sum_i (p_i - y_i)

The update is W_(t+1) = W_t - eta*dJ/dW and b_(t+1) = b_t - eta*dJ/db. These expressions are used in the from-scratch NumPy softmax-regression implementation. The experiment used standardised PCA-100 coordinates, regularisation lambda = 0.0001 and eight epochs.

8.2 Final losses and runtime

Strategy	Learning rate	Train loss (nats)	Validation loss (nats)	Gap
Batch GD	0.0010	2.3011	2.3011	0.0000
Batch GD	0.0100	2.2881	2.2880	-0.0001
Batch GD	0.1000	2.1673	2.1670	-0.0003
SGD	0.0010	1.5173	1.5219	0.0046
SGD	0.0100	1.6821	1.6883	0.0063
SGD	0.1000	6.3960	6.2345	-0.1614
Mini-batch SGD	0.0010	2.1680	2.1676	-0.0004
Mini-batch SGD	0.0100	1.6797	1.6756	-0.0041
Mini-batch SGD	0.1000	1.5187	1.5232	0.0045

Strategy	8-epoch runtime (s)	Runtime per epoch (s)
Batch gradient descent	7.3056	0.9132
Stochastic gradient descent	18.2309	2.2789
Mini-batch SGD	3.1803	0.3975

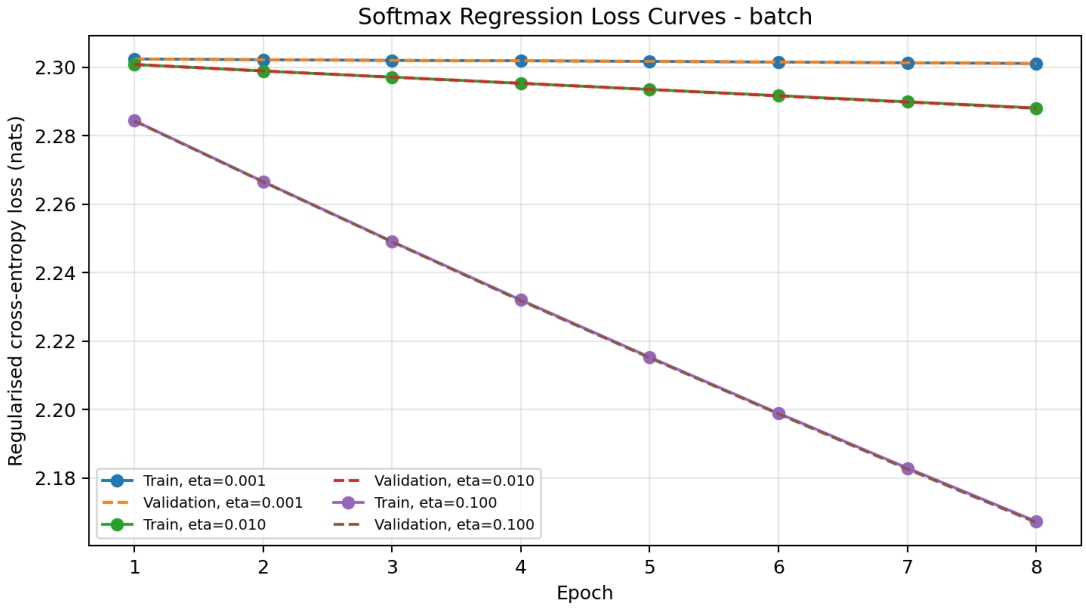


Figure 5. Batch gradient descent loss curves.

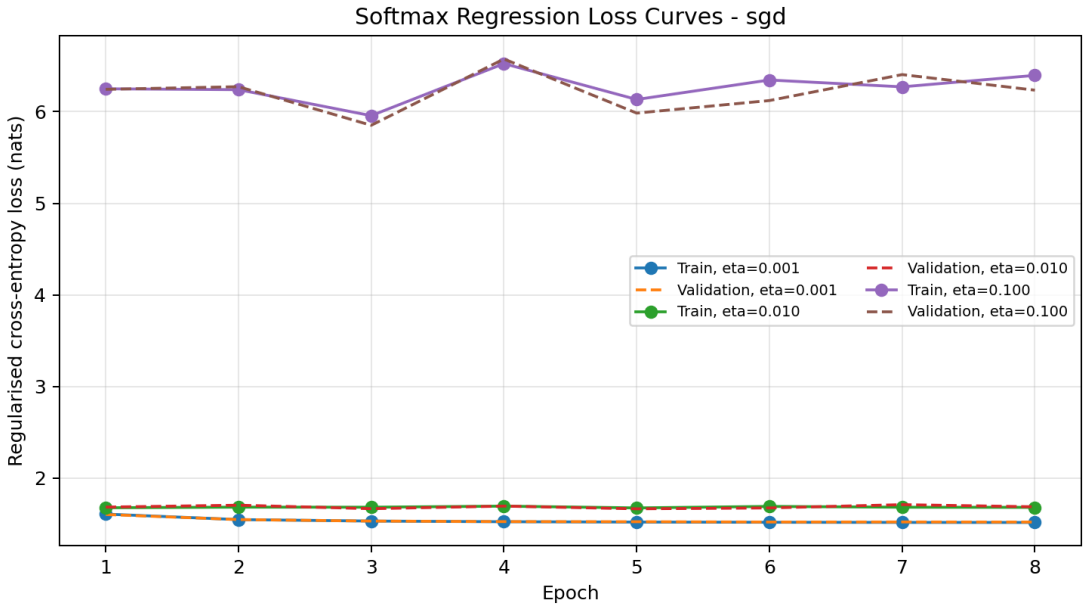


Figure 6. SGD loss curves; eta=0.1000 is visibly unstable in this run.

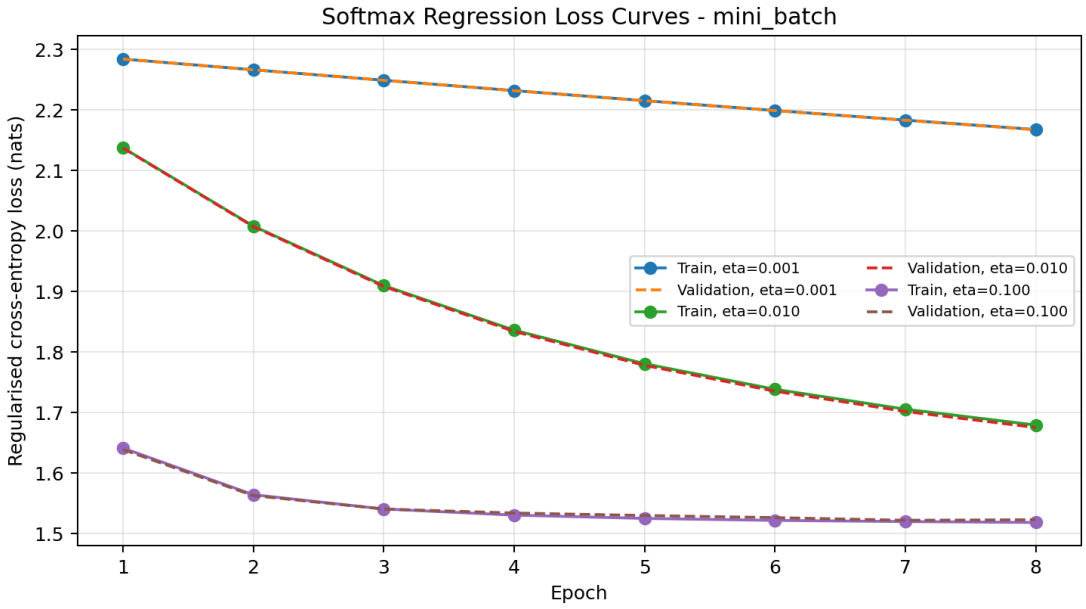


Figure 7. Mini-batch SGD loss curves.

SGD at eta=0.1000 fluctuates and ends with validation loss 6.2345, much worse than eta=0.0010. Mini-batch SGD reaches validation loss 1.5232 at eta=0.1000 in 3.1803 seconds for eight epochs. Batch GD is stable but changes slowly for small learning rates because this implementation makes one batch update per epoch.

The runtimes are measured per epoch for strategies with different numbers of updates; they are not an equal-update comparison. The observed behaviour is consistent with the need to tune learning rates and account for feature scaling/conditioning.Task 9 - Organise and examine experimental performance (3 marks)

9.1 Accuracy and macro-F1 across dataset scales

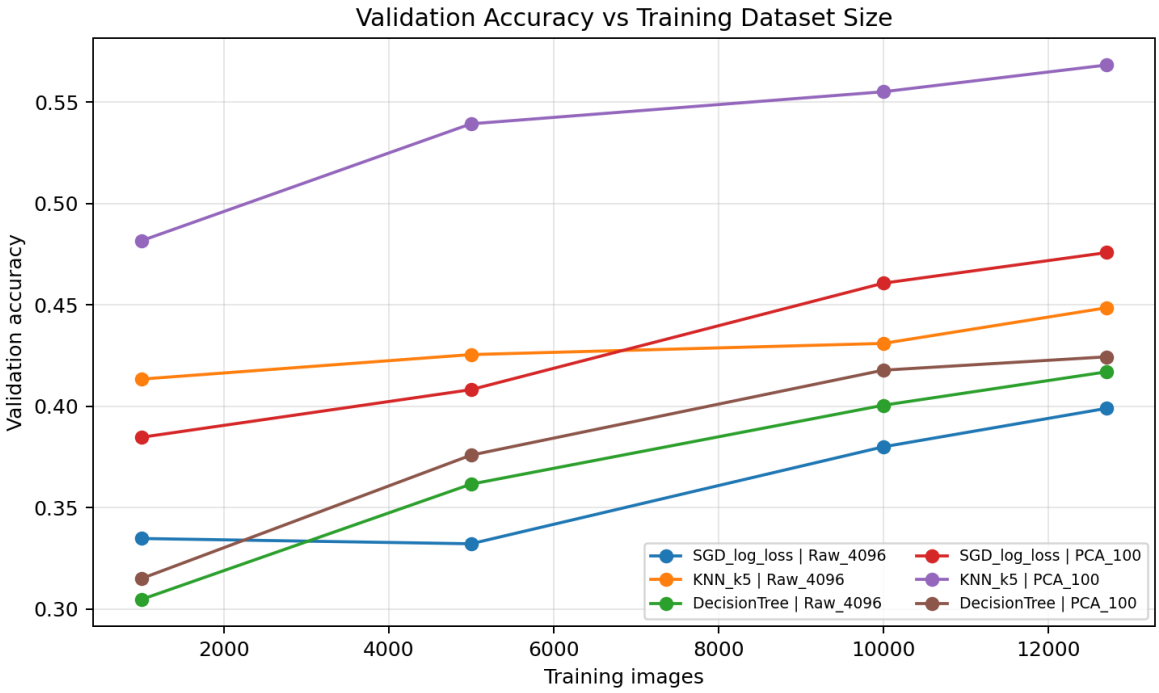


Figure 8. Validation accuracy for all models and dataset sizes.

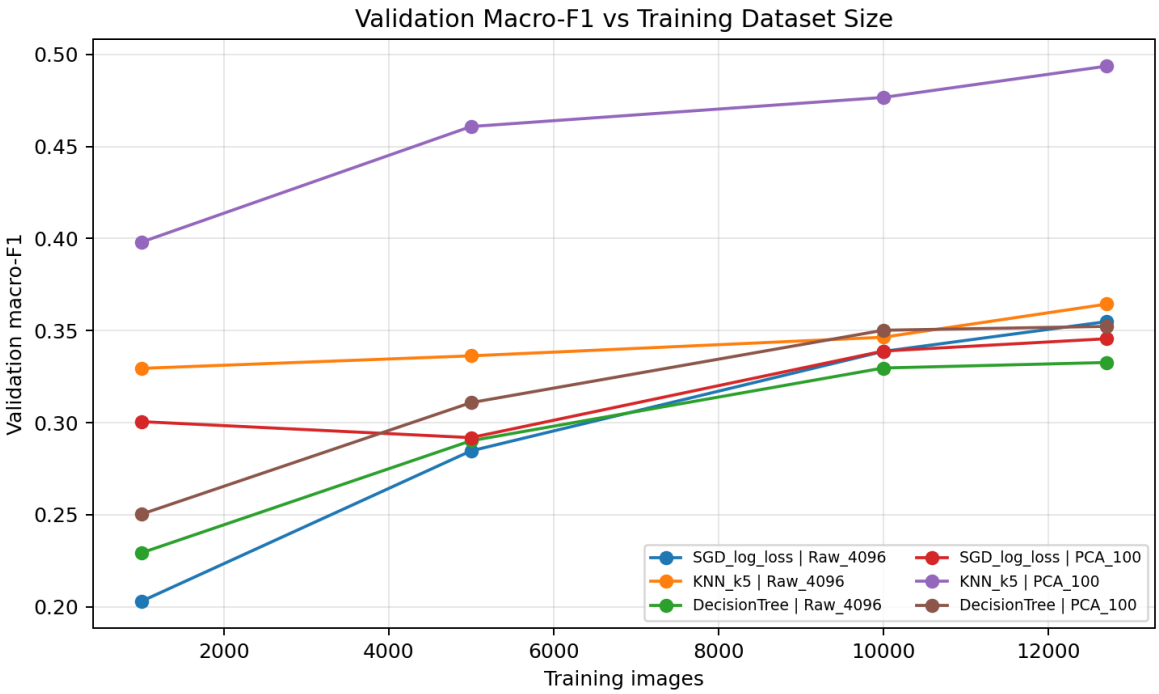


Figure 9. Validation macro-F1 for all models and dataset sizes.

For model size 12,712, k-NN with PCA-100 is best among the tested configurations at accuracy 0.5683 and macro-F1 0.4936. Its measured accuracy rises from 0.4816 at 1,000 training images to 0.5683 at 12,712. The other models also generally improve with more examples, although some metrics are not monotonic.

9.2 Raw-feature quality results

Training images	Model	Accuracy	Macro-F1
1000	SGD log loss	0.3348	0.2030
1000	k-NN	0.4134	0.3295
1000	Decision tree	0.3047	0.2293
5000	SGD log loss	0.3322	0.2847
5000	k-NN	0.4255	0.3363
5000	Decision tree	0.3616	0.2902
10000	SGD log loss	0.3800	0.3387
10000	k-NN	0.4310	0.3464
10000	Decision tree	0.4005	0.3297
12712	SGD log loss	0.3990	0.3549
12712	k-NN	0.4486	0.3644

Training images	Model	Accuracy	Macro-F1
12712	Decision tree	0.4170	0.3327

9.3 PCA-100 quality results

Training images	Model	Accuracy	Macro-F1
1000	SGD log loss	0.3847	0.3006
1000	k-NN	0.4816	0.3980
1000	Decision tree	0.3150	0.2503
5000	SGD log loss	0.4082	0.2919
5000	k-NN	0.5393	0.4608
5000	Decision tree	0.3759	0.3109
10000	SGD log loss	0.4607	0.3389
10000	k-NN	0.5551	0.4766
10000	Decision tree	0.4178	0.3502
12712	SGD log loss	0.4758	0.3456
12712	k-NN	0.5683	0.4936
12712	Decision tree	0.4244	0.3523

9.4 Full-scale cost and model-complexity measurements

The next two tables separate model quality from complexity because the count types differ between algorithms. All serialized sizes in these tables are decimal MB, computed as file bytes / 1,000,000.

n	Model	Fit s	Val predict s	Complexity count/type	Size MB
1000	SGD	13.5546	0.0164	40,970 coefficients	0.1651
1000	k-NN	0.0030	0.9246	4,097,000 stored values	16.3929
1000	Tree	5.2078	0.0073	387 nodes	0.0571
5000	SGD	77.5224	0.0177	40,970 coefficients	0.1651
5000	k-NN	0.0114	4.3628	20,485,000 stored values	81.9609
5000	Tree	28.7363	0.0068	1,757 nodes	0.2544
10000	SGD	164.5771	0.0936	40,970 coefficients	0.1651
10000	k-NN	0.0613	21.9737	40,970,000 stored values	163.9209

n	Model	Fit s	Val predict s	Complexity count/type	Size MB
10000	Tree	63.6722	0.0082	3,105 nodes	0.4485
12712	SGD	213.6941	0.0169	40,970 coefficients	0.1651
12712	k-NN	0.0376	12.2001	52,081,064 stored values	208.3760
12712	Tree	78.7767	0.0064	4,063 nodes	0.5865

n	Model	Fit s	Val predict s	Complexity count/type	Size MB
1000	SGD	0.3211	0.0014	1,010 coefficients	0.0053
1000	k-NN	0.0012	0.0347	101,000 stored values	0.4089
1000	Tree	0.1639	0.0014	477 nodes	0.0701
5000	SGD	1.9705	0.0038	1,010 coefficients	0.0053
5000	k-NN	0.0018	0.2726	505,000 stored values	2.0409
5000	Tree	1.6144	0.0015	2,021 nodes	0.2924
10000	SGD	3.3236	0.0012	1,010 coefficients	0.0053
10000	k-NN	0.0018	0.3006	1,010,000 stored values	4.0809
10000	Tree	2.6363	0.0012	3,893 nodes	0.5620
12712	SGD	3.7198	0.0012	1,010 coefficients	0.0053
12712	k-NN	0.0030	0.3923	1,283,912 stored values	5.1874
12712	Tree	4.2239	0.0012	4,889 nodes	0.7054

The counts are algorithm-specific proxies, not directly comparable parameter counts: SGD counts learned coefficients and intercepts; the tree counts nodes; k-NN counts stored feature values and labels. k-NN inference time and storage rise with the number of training examples. PCA-100 k-NN at full size stores about 5.1874 decimal MB, slightly above the 5 MB target, while its PCA transformation has a separate cost.

9.5 Confusion matrix and class-level behaviour

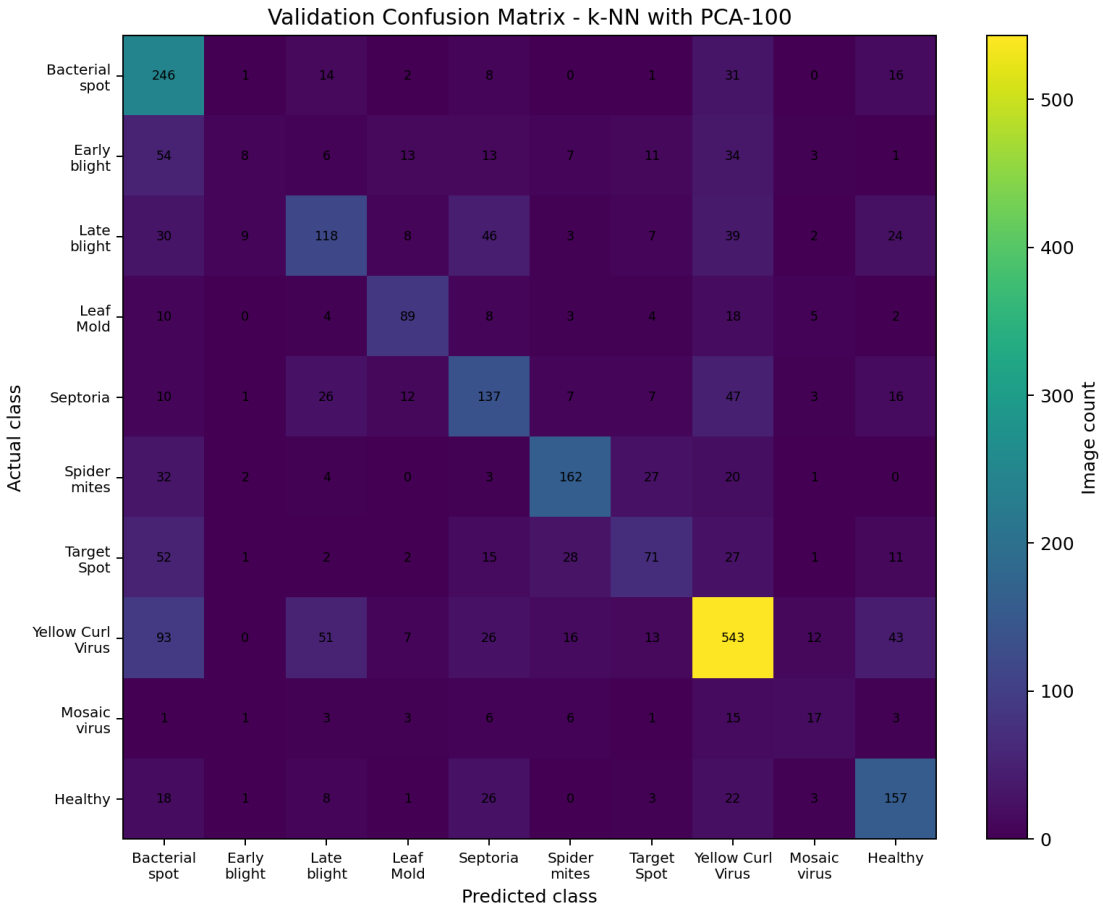


Figure 10. Confusion matrix for k-NN with PCA-100 on the 2,724-image tomato validation set.

Actual class	Predicted class	Count
Yellow Leaf Curl Virus	Bacterial Spot	93
Early Blight	Bacterial Spot	54
Target Spot	Bacterial Spot	52
Yellow Leaf Curl Virus	Late Blight	51
Septoria Leaf Spot	Yellow Leaf Curl Virus	47
Late Blight	Septoria Leaf Spot	46
Yellow Leaf Curl Virus	Healthy	43
Late Blight	Yellow Leaf Curl Virus	39
Early Blight	Yellow Leaf Curl Virus	34
Spider Mites	Bacterial Spot	32

Class	Precision	Recall	F1	Support

Class	Precision	Recall	F1	Support
Early Blight	0.3333	0.0533	0.0920	150
Tomato Mosaic Virus	0.3617	0.3036	0.3301	56
Target Spot	0.4897	0.3381	0.4000	210
Late Blight	0.5000	0.4126	0.4521	286
Septoria Leaf Spot	0.4757	0.5150	0.4946	266
Leaf Mold	0.6496	0.6224	0.6357	143
Spider Mites	0.6983	0.6454	0.6708	251
Healthy	0.5751	0.6569	0.6133	239
Yellow Leaf Curl Virus	0.6822	0.6754	0.6788	804
Bacterial Spot	0.4505	0.7712	0.5688	319

The most serious class-level weakness is Early Blight recall: 8 of 150 Early Blight validation images are correct, giving 8 / 150 = 0.0533. Only 8 of 24 images predicted as Early Blight are correct, giving precision 8 / 24 = 0.3333. This is why accuracy alone is inadequate for disease screening.

Task 10 - Compare theoretical expectations with experimental behaviour (3 marks)

Expectation	Measured evidence	Interpretation
Variance retained does not determine class accuracy exactly	PCA-100 retains 81.8237% variance for the tomato training set; k-NN accuracy rises from 0.4486 raw to 0.5683 after PCA	PCA retains variance, not necessarily class-discriminative information; classification must be measured directly
Model conditioning and learning rates influence convergence	PCA-256 covariance condition number 789.2001; SGD at eta=0.1000 gave validation loss 6.2345 with fluctuations	Optimisation is sensitive to step size; Task 8 standardised its PCA coordinates, so it is not a direct unstandardised conditioning test
High capacity can increase variance and generalisation gap	Depth-32 tree gap 0.4768; bootstrap variance 0.5137 versus 0.0027 at depth 2	Both experiments indicate high-capacity tree predictions are more sensitive to training-sample choice
A model can match marginal class frequencies without good predictions	Raw decision tree KL(p||q)=0.0010 but accuracy 0.4170 and per-image CE=20.7338	Marginal probability distribution comparisons do not replace per-example loss and classification metrics

Overall, theory and experiments broadly agree, but they answer different questions. Reconstruction variance concerns image variation; classification accuracy concerns separation of labels; covariance conditioning concerns optimisation geometry; and bias-variance experiments concern sensitivity to the sampled training data.

Task 11 - Analyse whether deep learning is justified (4 marks)

11.1 High dimensionality and classical-model limitations

Samples per raw feature = N / d = 12712 / 4096 = 3.1035

The raw training matrix has only 3.1035 training samples per input feature. This illustrates the high-dimensional setting in which distance-based models can struggle to find useful neighbours and linear models may not capture complex image patterns.

PCA-100 reduces the representation by 97.5586% and improves k-NN accuracy by 0.1197 (11.9700 percentage points), yet the best baseline reaches only 0.5683 accuracy and 0.4936 macro-F1. Early Blight recall is 0.0533. Thus, the best tuned baseline still has a class-specific failure that matters for a screening application.

11.2 Local constancy, smoothness and manifold structure

k-NN follows a local-constancy assumption: nearby feature vectors should share labels. PCA-100 improved its result, suggesting that the lower-dimensional metric space is more useful for this model on this dataset. Nevertheless, flattening loses the explicit 2-D pixel arrangement, and PCA captures linear directions rather than learned nonlinear image features. Disease patterns may depend on spots, texture, edges and spatial relationships across the leaf.

A compact convolutional neural network could learn local features and combine them into higher-level patterns. This is a reason to test deep learning against the current baselines. No CNN was trained in the recorded experiments, so the report does not claim that deep learning has already achieved higher accuracy. The evidence justifies a controlled CNN experiment, not a direct claim of superiority.

11.3 Trade-offs

Classical models are easier to inspect and some are small, but the current results are weak. Deep learning may learn richer image representations but usually requires additional training and deployment work. The next comparison should use the same fixed split, report macro-F1 and per-class recall, and measure actual deployment size and single-image latency.

Task 12 - Analyse the impact of changed requirements (8 marks)

A. Impact analysis (2 marks)

The problem changes from 10 tomato classes to 38 crop-disease classes and introduces 20% noisy training labels. The 38-class training split contains 38,013 images, validation contains 8,146 and test contains 8,146. The dataset was converted to 64 x 64 grayscale (4,096 features), and PCA-100 retained 85.2584% of training variance.

Maximum entropy for K classes: H_max = log2(K)

For K = 38, H_max = log2(38) = 5.2479 bits. Measured class entropy is 4.8249 bits for training and 4.8252 bits for validation; each has normalised entropy 0.9194. The most common validation class is Orange___Haunglongbing_(Citrus_greening), with 826 of 8,146 images, so the majority baseline accuracy is 0.1014.

The noise generator changed 7,603 of 38,013 training labels, giving a measured noise fraction of 20.0011%. Validation and test labels were unchanged. The class distribution entropy and majority baseline depend on the label distribution, but sensitivity/specificity concepts from binary classification are not the metric used for this 38-class experiment.

B. Re-examination of the original conclusion (2 marks)

Configuration	Validation accuracy	Validation macro-F1	Saved pipeline MB	Kaggle p95 latency (ms)
SGD log loss, clean, PCA-100	0.3712	0.2645	1.5524	0.5641
SGD log loss, noisy, PCA-100	0.3520	0.2344	1.5522	0.3858
Modified Huber, noisy, PCA-100	0.2615	0.1582	1.5522	0.3984
Modified Huber, noisy, PCA-64	0.2701	0.1556	0.9986	0.5357
k-NN, noisy, PCA-100	0.3679	0.2510	15.7618	11.5936

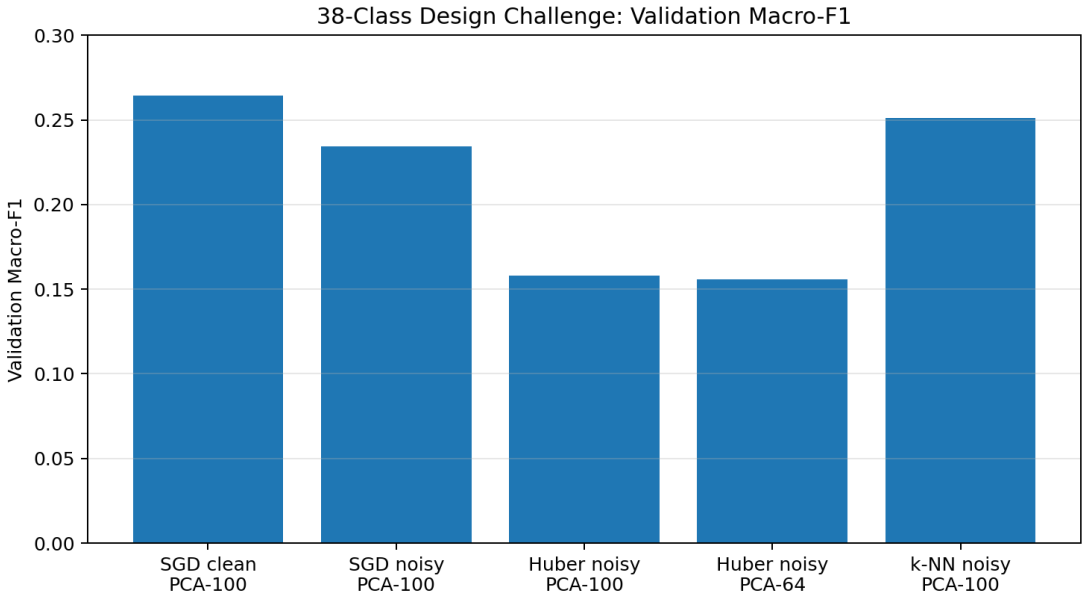


Figure 11. Validation Macro-F1 for the five 38-class configurations.

The noisy SGD model loses 0.3712 - 0.3520 = 0.0192 accuracy after the training labels are corrupted, and macro-F1 falls from 0.2645 to 0.2344. Four SGD configurations fit the measured artifact-size and Kaggle p95 proxies; k-NN exceeds the 5 MB file-size proxy at 15.7618 MB. However, these are saved-file size and Kaggle latency proxies, not proof of target-phone RAM or latency.

C. Adaptation options (2 marks)

Option 1 - modified Huber loss: for PCA-100 with noisy labels, validation accuracy is 0.2615 compared with 0.3520 for log-loss SGD, a difference of -0.0905. Macro-F1 also falls from 0.2344 to 0.1582. The selected modified-Huber configuration did not help in this experiment.

Option 2 - reduce PCA dimension to 64: the modified-Huber artifact falls from 1.5522 MB at 100 components to 0.9986 MB at 64 components. The relative saved-file reduction is (1.5522 - 0.9986) / 1.5522 x 100 = 35.6640%. Macro-F1 is 0.1556 for PCA-64, slightly lower than 0.1582 for PCA-100, so this smaller setting saves space but did not improve class-balanced performance.

Option 3 - replace k-NN with a linear classifier for deployment: k-NN has better validation accuracy than noisy SGD (0.3679 vs 0.3520) but its saved pipeline is 15.7618 MB, above the 5 MB target. Noisy SGD is much smaller and faster in the Kaggle proxy, but its predictive performance remains inadequate.

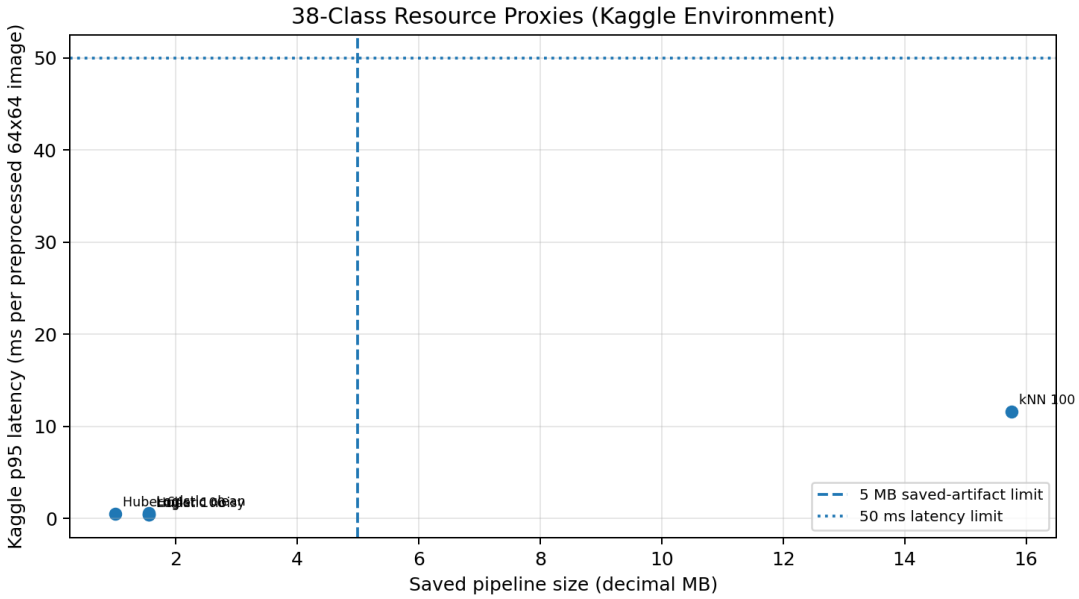


Figure 12. Saved pipeline size and Kaggle p95 latency proxy; phone performance must be measured separately.

D. Evidence-based conclusion (2 marks)

The selected noisy-label configuration is SGD with log loss and PCA-100, chosen as the highest validation Macro-F1 among the tested noisy-label configurations that met both measured proxies. Its validation accuracy is 0.3520 and Macro-F1 is 0.2344. The held-out test accuracy is 0.3451 and test Macro-F1 is 0.2266. The saved pipeline is 1.5522 decimal MB, and its p95 timing proxy is 0.3858 ms per already-resized 64 x 64 image in Kaggle.

Final Task 12 decision: redesign before deployment. The measurements support the storage and timing proxy of the selected compact SGD pipeline, but classification quality is too low for dependable screening, and the optimiser reported convergence warnings. The next iteration should address label quality, train to convergence, evaluate compact learned image features (including a small CNN), then benchmark complete end-to-end inference and peak memory on a representative low-cost smartphone. Because that device benchmark was not performed here, the 50 ms / 5 MB requirement is not yet conclusively verified.

Overall conclusion and remaining limitations

The experiments show that representation, optimisation, capacity and class distribution materially affect crop-disease classification. PCA-100 substantially reduces feature count and improves the tested k-NN baseline; nonetheless, class-level errors and low macro-F1 remain. The high-capacity decision tree overfits in the tested settings, and the bootstrap experiment shows much greater variance for the depth-32 tree. The 38-class label-noise experiment further reduces the quality of a compact SGD model.

The evidence supports a redesign rather than deployment of the current system. Two limitations must be carried into any final submission: (1) several SGD fits reached their iteration limit, so these comparisons should be rerun with convergence addressed; and (2) target-smartphone latency and peak memory were

not measured. Also, the PCA component-image figure should be inspected in Kaggle so its visual interpretation can be completed accurately. No CNN benchmark was performed, so deep learning is a justified next experiment rather than a proven winner in this report.

Appendix A - Captured Kaggle output records

The following records summarise key printed values from the Kaggle cells shared during the assessment workflow. They are included to preserve the calculation evidence next to the reproducibility code. Runtimes may vary when cells are rerun.

A1. Dataset and preprocessing outputs

```
TOTAL DATASET: classes=38; images=54305
TOMATO EXPERIMENT: classes=10; images=18160
tomato_train=(12712,4096); tomato_val=(2724,4096); tomato_test=(2724,4096)
38-class train=(38013,4096); validation=(8146,4096); test=(8146,4096)
One image=(64,64); flattened image=(4096,); batch of 32=(32,4096)
First image norms: L1=1631.4549; L2=27.6263; Linfinity=0.6824
NumPy norm verification passed for all three norms.
```

A2. PCA outputs

k	Explained variance (%)	Relative reconstruction error
5	52.1264	0.7058
10	60.0481	0.6483
20	67.1677	0.5885
50	75.8431	0.5063
100	81.8237	0.4433
150	85.1243	0.4036
200	87.3225	0.3748
256	89.0371	0.3510

```
2x2 covariance C=[[0.02932759,0.02667736],[0.02667736,0.02894818]]
Manual eigenvalues: lambda_1=0.0558; lambda_2=0.0025
Eigenvectors (columns): [[-0.7096,0.7046],[-0.7046,-0.7096]]
Frobenius decomposition error: 0.0000000000; NumPy verification passed.
PCA 64-component fit times (s): n=1000:2.7361; n=5000:4.4388; n=10000:4.4004; n=12712:5.7493
```

A3. Numerical stability output

```
Logits: [1000.0, 999.0, 995.0]
Naive exponentials: [inf, inf, inf]; naive probabilities: [nan, nan, nan]
Stable shifted logits: [0.0, -1.0, -5.0]
```

Stable probabilities: [0.7275, 0.2676, 0.0049]; sum=1.0000
Stable log-sum-exp=1000.3182; retained PCA-256 condition number=789.2001

A4. Capacity / bootstrap / MLE-MAP output

Capacity	Depth	Train error	Validation error	Gap
C1	2	0.6573	0.6553	-0.0020
C2	4	0.5753	0.5815	0.0062
C3	8	0.4541	0.5209	0.0668
C4	16	0.1340	0.5734	0.4394
C5	32	0.0944	0.5712	0.4768

Model	Bias-squared proxy	Variance	Mean Brier loss
Depth 2	0.7851	0.0027	0.7878
Depth 32	0.6166	0.5137	1.1303

n	s	MLE	MAP Beta(2,2)	Absolute difference
100	28	0.2800	0.2843	0.0043
500	155	0.3100	0.3108	0.0008
1000	279	0.2790	0.2794	0.0004
5000	1451	0.2902	0.2903	0.0001
12712	3750	0.2950	0.2950	0.0000

A5. Optimisation final output

Strategy	eta	Train loss	Validation loss	Gap
Batch	0.0010	2.3011	2.3011	0.0000
Batch	0.0100	2.2881	2.2880	-0.0001
Batch	0.1000	2.1673	2.1670	-0.0003
SGD	0.0010	1.5173	1.5219	0.0046
SGD	0.0100	1.6821	1.6883	0.0063
SGD	0.1000	6.3960	6.2345	-0.1614
Mini-batch	0.0010	2.1680	2.1676	-0.0004

Strategy	eta	Train loss	Validation loss	Gap
Mini-batch	0.0100	1.6797	1.6756	-0.0041
Mini-batch	0.1000	1.5187	1.5232	0.0045

A6. Task 12 output

38-class training entropy=4.8249 bits; validation entropy=4.8252 bits; max entropy=5.2479 bits.
Normalised entropy=0.9194; majority class Orange___Haunglongbing_(Citrus_greening).
Majority baseline accuracy=0.1014 for training and validation.
Noisy training labels changed=7603/38013; measured noise rate=20.0011%.
PCA100 cumulative explained variance=85.2584%.
Selected noisy-label SGD log-loss PCA100: validation accuracy=0.3520; validation macro-F1=0.2344.
Selected held-out test accuracy=0.3451; test macro-F1=0.2266.
Saved pipeline size=1.5522 decimal MB; Kaggle p95 latency proxy=0.3858 ms per already-resized image.
Models emitted convergence warnings when maximum iteration count was reached.

Appendix B - Consolidated reproducibility code

The code listing below consolidates the main preprocessing, verification and experimental cells into one Kaggle-oriented source file. It expects the PlantVillage dataset to be attached at the path shown in the code or the processed arrays saved under /kaggle/working/agriVision. Captured outputs from the user-executed cells are recorded in Appendix A. The code has been syntax-checked locally but has not been rerun against the Kaggle dataset in this authoring environment.

Source listing: AgriVision_Assessment1_Code.py (618 lines).

```
0001  # CSA47 Deep Learning - CO1 Assessment 1 (AgriVision)
0002  # Consolidated reproducibility code for Kaggle Notebooks.
0003  #
0004  # Prerequisites:
0005  # 1. Attach the PlantVillage dataset in Kaggle.
0006  # 2. Run the dataset-manifest, split and preprocessing cells from the workflow,
0007  #    or create the required arrays in /kaggle/working/agriVision.
0008  # 3. Set OUT to the directory containing those arrays.
0009  #
0010  # Outputs reported in the accompanying assessment report are captured from the
0011  # user's Kaggle execution. Timing values may vary on rerun and should not be
0012  # expected to match exactly across notebook sessions or hardware.
0013
0014  from pathlib import Path
0015  from time import perf_counter, perf_counter_ns
0016  import json
0017  import gc
0018  import warnings
0019
0020  import numpy as np
0021  import pandas as pd
0022  import matplotlib.pyplot as plt
0023  import joblib
0024
0025  from sklearn.decomposition import PCA
0026  from sklearn.model_selection import train_test_split
0027  from sklearn.preprocessing import StandardScaler
0028  from sklearn.linear_model import SGDClassifier
0029  from sklearn.neighbors import KNeighborsClassifier
0030  from sklearn.tree import DecisionTreeClassifier
0031  from sklearn.metrics import (
```

```
    accuracy_score, precision_recall_fscore_support,
    confusion_matrix, f1_score
)
from sklearn.exceptions import ConvergenceWarning

OUT = Path('/kaggle/working/agriVision')
OUT.mkdir(parents=True, exist_ok=True)
IMAGE_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.bmp'}
RANDOM_STATE = 42

# %% [markdown]
# DATASET MANIFEST AND PREPROCESSING
# This cell is needed if the manifests / processed arrays do not already exist.

# %%
DATA_ROOT = Path('/kaggle/input/plantvillage-dataset/plantvillage dataset')
COLOR_DIR = DATA_ROOT / 'color'

if COLOR_DIR.is_dir():
    records = []
    for class_dir in sorted(COLOR_DIR.iterdir()):
        if not class_dir.is_dir():
            continue
        for image_path in class_dir.iterdir():
            if image_path.is_file() and image_path.suffix.lower() in IMAGE_EXTENSIONS:
                records.append({
                    'path': str(image_path),
                    'label': class_dir.name,
                    'crop_group': 'tomato' if class_dir.name.startswith('Tomato___') else 'other'
                })
    manifest = pd.DataFrame(records)
    if manifest.empty:
        raise ValueError('No color image files were found.')
    manifest.to_csv(OUT / 'manifest_all38.csv', index=False)
    tomato_df = manifest[manifest['crop_group'] == 'tomato'].reset_index(drop=True)
    all38_df = manifest.reset_index(drop=True)
    print('Color dataset classes:', manifest['label'].nunique())
```

```
    print('Color dataset images:', len(manifest))
    print('Tomato images:', len(tomato_df))
else:
    print('Color source folder not found. Reuse existing saved arrays if available.')

# 70/15/15 stratified splits. This is an experimental choice, not a prescribed
# split in the assessment.
def make_splits(frame, seed=RANDOM_STATE):
    train_val, test = train_test_split(
        frame, test_size=0.15, random_state=seed, stratify=frame['label']
    )
    train, val = train_test_split(
        train_val, test_size=(0.15 / 0.85), random_state=seed + 1,
        stratify=train_val['label']
    )
    return tuple(x.reset_index(drop=True) for x in (train, val, test))


def preprocess_frame(frame, label_to_id, split_name, size=(64, 64)):
    from PIL import Image
    from tqdm.auto import tqdm
    X = np.empty((len(frame), size[0] * size[1]), dtype=np.uint8)
    y = np.empty(len(frame), dtype=np.int64)
    for i, row in enumerate(tqdm(frame.itertuples(index=False), total=len(frame), desc=split_name)):
        try:
            with Image.open(row.path) as im:
                gray = im.convert('L').resize(size, Image.Resampling.LANCZOS)
                X[i] = np.asarray(gray, dtype=np.uint8).reshape(-1)
            y[i] = label_to_id[row.label]
        except Exception as exc:
            raise RuntimeError(f'Failed image: {row.path}') from exc
    return X, y

# If the arrays already exist, do not rebuild unnecessarily.
required_tomato = [
    'X_train_uint8.npy', 'y_train.npy', 'X_val_uint8.npy', 'y_val.npy',
    'X_test_uint8.npy', 'y_test.npy', 'tomato_class_names.json'
]
```

```
if not all((OUT / f).is_file() for f in required_tomato) and 'tomato_df' in globals():
    tomato_train, tomato_val, tomato_test = make_splits(tomato_df)
    tomato_class_names = sorted(tomato_df['label'].unique())
    tomato_label_to_id = {name: i for i, name in enumerate(tomato_class_names)}
    for split_name, frame in [('train', tomato_train), ('val', tomato_val), ('test', tomato_test)]:
        X, y = preprocess_frame(frame, tomato_label_to_id, f'tomato_{split_name}')
        np.save(OUT / f'X_{split_name}_uint8.npy', X)
        np.save(OUT / f'y_{split_name}.npy', y)
    with open(OUT / 'tomato_class_names.json', 'w', encoding='utf-8') as f:
        json.dump(tomato_class_names, f, indent=2)

# Build 38-class arrays when they are not already saved.
required_38 = [
    'X38_train_uint8.npy', 'y38_train.npy', 'X38_val_uint8.npy',
    'y38_val.npy', 'X38_test_uint8.npy', 'y38_test.npy',
    'all38_class_names.json'
]
if not all((OUT / f).is_file() for f in required_38) and 'all38_df' in globals():
    all38_train, all38_val, all38_test = make_splits(all38_df)
    class_names_38 = sorted(all38_df['label'].unique())
    label_to_id_38 = {name: i for i, name in enumerate(class_names_38)}
    for split_name, frame in [('train', all38_train), ('val', all38_val), ('test', all38_test)]:
        X, y = preprocess_frame(frame, label_to_id_38, f'all38_{split_name}')
        np.save(OUT / f'X38_{split_name}_uint8.npy', X)
        np.save(OUT / f'y38_{split_name}.npy', y)
    with open(OUT / 'all38_class_names.json', 'w', encoding='utf-8') as f:
        json.dump(class_names_38, f, indent=2)

# %% [markdown]
# TASK 1 - REPRESENTATION AND NORMS
```

```

# %%
X_train_u8 = np.load(OUT / 'X_train_uint8.npy')
y_train = np.load(OUT / 'y_train.npy')
y_val = np.load(OUT / 'y_val.npy')
y_test = np.load(OUT / 'y_test.npy')
```

```
with open(OUT / 'tomato_class_names.json', encoding='utf-8') as f:
    tomato_class_names = json.load(f)

x = X_train_u8[0].astype(np.float64) / 255.0
L1 = np.sum(np.abs(x))
L2 = np.sqrt(np.sum(x ** 2))
Linf = np.max(np.abs(x))
assert np.isclose(L1, np.linalg.norm(x, ord=1))
assert np.isclose(L2, np.linalg.norm(x, ord=2))
assert np.isclose(Linf, np.linalg.norm(x, ord=np.inf))
print('TASK 1 NORMS')
print('Features:', len(x), 'L1:', f'{L1:.4f}', 'L2:', f'{L2:.4f}', 'Linf:', f'{Linf:.4f}')
print('Shapes: image=(64,64), vector=(4096,), batch=(32,4096), training=', X_train_u8.shape)

# %% [markdown]
# TASK 2 - PCA / EIGENVALUES / RECONSTRUCTION

# %%
X_train = X_train_u8.astype(np.float32) / 255.0
X_val_u8 = np.load(OUT / 'X_val_uint8.npy')
X_val = X_val_u8.astype(np.float32) / 255.0

pca_file = OUT / 'tomato_pca_256.joblib'
if pca_file.exists():
    pca = joblib.load(pca_file)
else:
    pca = PCA(n_components=256, svd_solver='randomized', random_state=42, iterated_power=4)
    t0 = perf_counter(); pca.fit(X_train); print('PCA fit seconds:', f'{perf_counter()-t0:.4f}')
    joblib.dump(pca, pca_file)

singular_values = pca.singular_values_
eigenvalues = singular_values ** 2 / (len(X_train) - 1)
cumulative = np.cumsum(pca.explained_variance_ratio_)
print('First ten singular values:', np.round(singular_values[:10], 4))
print('First ten covariance eigenvalues:', np.round(eigenvalues[:10], 4))

ks = np.array([5, 10, 20, 50, 100, 150, 200, 256])
rng = np.random.default_rng(42)
```

```
ids = rng.choice(len(X_val), size=min(200, len(X_val)), replace=False)
X_eval = X_val[ids]
Xc = X_eval - pca.mean_
Z = pca.transform(X_eval)
recon_rows = []
for k in ks:
    reconstructed = Z[:, :k] @ pca.components_[:k]
    err = np.linalg.norm(Xc - reconstructed, ord='fro') / np.linalg.norm(Xc, ord='fro')
    recon_rows.append({'k': int(k), 'explained_variance_pct': 100 * cumulative[k-1], 'relative_reconstruction_error': err})
recon_df = pd.DataFrame(recon_rows)
recon_df.to_csv(OUT / 'pca_reconstruction_results.csv', index=False)
print(recon_df.round(4).to_string(index=False))

fig, ax = plt.subplots(figsize=(7.5, 4.2)); ax.plot(ks, 100*cumulative[ks-1], marker='o')
ax.set_title('Cumulative Explained Variance vs PCA Components'); ax.set_xlabel('Components k'); ax.set_ylabel('Explained variance (%)'); ax.grid(True, alpha=.3); fig.tight_layout(); fig.savefig(OUT/'pca_explained_variance.png', dpi=180); plt.show()
fig, ax = plt.subplots(figsize=(7.5, 4.2)); ax.plot(ks, recon_df['relative_reconstruction_error'], marker='o')
ax.set_title('Validation Reconstruction Error vs PCA Components'); ax.set_xlabel('Components k'); ax.set_ylabel('Relative Frobenius error'); ax.grid(True, alpha=.3); fig.tight_layout(); fig.savefig(OUT/'pca_reconstruction_error.png', dpi=180); plt.show()

# Visualise leading principal components as 64 x 64 coefficient images.
fig, axes = plt.subplots(2, 3, figsize=(9, 6))
for pc, ax in enumerate(axes.flat):
    ax.imshow(pca.components_[pc].reshape(64, 64), cmap='coolwarm')
    ax.set_title(f'PC {pc+1}'); ax.axis('off')
fig.suptitle('Leading PCA Components of Tomato Leaf Images')
```

```
fig.tight_layout(); fig.savefig(OUT/'pca_component_images.png', dpi=180); plt.show()

# Measure PCA fitting cost at the required dataset scales for 64 components.
pca_timing_rows = []
for sample_n in sorted(set([1000, 5000, 10000, len(X_train)])):
    if sample_n == len(X_train): X_sub = X_train
    else: X_sub = X_train[np.random.default_rng(42+sample_n).choice(len(X_train), size=sample_n, replace=False)]
    timed_pca = PCA(n_components=64, svd_solver='randomized', random_state=42, iterated_power=3)
    t0 = perf_counter(); timed_pca.fit(X_sub); elapsed = perf_counter()-t0
    pca_timing_rows.append({'Training_images': sample_n, 'Components': 64, 'Fit_time_seconds': elapsed,
```

```
                        'Explained_variance_pct': 100*np.sum(timed_pca.explained_variance_ratio_)})
pca_time_df = pd.DataFrame(pca_timing_rows)
pca_time_df.to_csv(OUT/'pca_fit_time_by_dataset_size.csv', index=False)
print('PCA fitting times by sample size:')
print(pca_time_df.round(4).to_string(index=False))

# A real 2x2 covariance subproblem; eigenvalues and vectors are checked by NumPy.
variances = np.var(X_train, axis=0)
i = int(np.argmax(variances)); ci = X_train[:, i] - X_train[:, i].mean()
covs = ((X_train - X_train.mean(axis=0)).T @ ci) / (len(X_train) - 1); covs[i] = 0
j = int(np.argmax(np.abs(covs)))
C = np.cov(X_train[:, [i, j]], rowvar=False, ddof=1)
vals, Q = np.linalg.eigh(C); order = np.argsort(vals)[::-1]; vals = vals[order]; Q = Q[:, order]
print('2x2 covariance matrix:\n', C)
print('Eigenvalues:', np.round(vals, 4)); print('Eigenvectors (columns):\n', np.round(Q, 4))
print('Reconstruction Frobenius error:', f'{np.linalg.norm(C-Q@np.diag(vals)@Q.T, ord="fro"):.10f}')

# %% [markdown]
# TASKS 3 AND 5 - BASELINE MODELS AND INFORMATION THEORY

# %%
from sklearn.metrics import log_loss

def build_model(name):
    if name == 'SGD_log_loss':
        return SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=100, tol=1e-3, random_state=42)
    if name == 'KNN_k5':
        return KNeighborsClassifier(n_neighbors=5, weights='distance', algorithm='brute', n_jobs=-1)
    if name == 'DecisionTree':
        return DecisionTreeClassifier(max_depth=20, min_samples_leaf=2, random_state=42)
    raise ValueError(name)

def info_metrics(y_true, probs, n_classes):
    eps = 1e-12
    p = np.bincount(y_true, minlength=n_classes).astype(np.float64) / len(y_true)
    q = np.clip(probs.mean(axis=0), eps, None); q /= q.sum()
    mask = p > 0
    H = -np.sum(p[mask] * np.log2(p[mask]))
```

```
    Hpq = -np.sum(p[mask] * np.log2(q[mask]))
    KLpq = np.sum(p[mask] * np.log2(p[mask] / q[mask]))
    KLqp = np.sum(q[mask] * np.log2(q[mask] / p[mask]))
    return {'H_true_bits': H, 'marginal_cross_entropy_bits': Hpq,
            'KL_p_to_q_bits': KLpq, 'KL_q_to_p_bits': KLqp,
            'identity_gap': abs(Hpq - (H + KLpq))}

# Uses PCA-100 and raw representations, matching the original experiment.
Z_train100 = pca.transform(X_train)[:, :100].astype(np.float32)
Z_val100 = pca.transform(X_val)[:, :100].astype(np.float32)
results = []
for rep, (Xa, Xv) in {
    'Raw_4096': (X_train, X_val),
    'PCA_100': (Z_train100, Z_val100)
}.items():
    for name in ['SGD_log_loss', 'KNN_k5', 'DecisionTree']:
        model = build_model(name); t0 = perf_counter()
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter('always', ConvergenceWarning)
            model.fit(Xa, y_train)
```

```
        fit_s = perf_counter() - t0
        t0 = perf_counter(); preds = model.predict(Xv); predict_s = perf_counter() - t0
        accuracy = accuracy_score(y_val, preds)
        prec, rec, f1, _ = precision_recall_fscore_support(y_val, preds, average='macro', zero_division=0)
        row = {'Model': name, 'Representation': rep, 'Accuracy': accuracy, 'Macro_precision': prec, 'Macro_recall': rec, 'Macro_F1': f1,
'Training_time_seconds': fit_s, 'Validation_prediction_seconds': predict_s}
        if hasattr(model, 'predict_proba'):
            probs = model.predict_proba(Xv)
            aligned = np.zeros((len(Xv), len(tomato_class_names)), dtype=float)
            for idx, cls in enumerate(model.classes_): aligned[:, int(cls)] = probs[:, idx]
            row.update(info_metrics(y_val, aligned, len(tomato_class_names)))
        results.append(row)
        joblib.dump(model, OUT / f'{name}_{rep}.joblib')
        print(name, rep, 'accuracy=', f'{accuracy:.4f}', 'macroF1=', f'{f1:.4f}', 'fit_s=', f'{fit_s:.4f}', 'predict_s=', f'{predict_s:.4f}')
        del model; gc.collect()
baselines = pd.DataFrame(results)
```

```
baselines.to_csv(OUT / 'baseline_reproduction_results.csv', index=False)
print(baselines.round(4).to_string(index=False))

# %% [markdown]
# TASK 4 - NUMERICAL STABILITY AND CONDITIONING

# %%
z = np.array([1000.0, 999.0, 995.0])
with np.errstate(over='ignore', invalid='ignore', divide='ignore'):
    naive = np.exp(z) / np.exp(z).sum()
m = np.max(z); stable_exp = np.exp(z-m); stable = stable_exp / stable_exp.sum()
lse = m + np.log(stable_exp.sum())
print('Naive softmax:', naive)
print('Stable probabilities:', np.round(stable, 4), 'sum=', f'{stable.sum():.4f}')
print('Stable log-sum-exp:', f'{lse:.4f}')
assert np.all(np.isfinite(stable)) and np.isclose(stable.sum(), 1)
condition_number = pca.explained_variance_[0] / pca.explained_variance_[-1]
print('Condition number retained PCA-256 covariance subspace:', f'{condition_number:.4f}')

# %% [markdown]
# TASK 6 - CAPACITY CURVE (also maps to the extra Part (e) plot instruction)

# %%
cap_rows = []
capacities = {'C1': 2, 'C2': 4, 'C3': 8, 'C4': 16, 'C5': 32}
for label, depth in capacities.items():
    model = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=2, random_state=42)
    t0 = perf_counter(); model.fit(Z_train100, y_train); fit_s = perf_counter() - t0
    tr_error = 1 - accuracy_score(y_train, model.predict(Z_train100))
    va_error = 1 - accuracy_score(y_val, model.predict(Z_val100))
    cap_rows.append({'Capacity': label, 'Depth': depth, 'Training_error': tr_error,
                     'Validation_error': va_error, 'Generalisation_gap': va_error-tr_error, 'Fit_time': fit_s})
cap_df = pd.DataFrame(cap_rows)
cap_df.to_csv(OUT / 'capacity_error_results.csv', index=False)
print(cap_df.round(4).to_string(index=False))
best_idx = int(np.argmin(cap_df['Validation_error'].to_numpy()))
fig, ax = plt.subplots(figsize=(8, 5))
ax.axvspan(-.5, 1.5, alpha=.10, label='Underfitting region (C1-C2)')
```

```
ax.axvspan(2.5, 4.5, alpha=.08, label='Overfitting region (C4-C5)')
ax.plot(range(5), cap_df['Training_error'], marker='o', label='Training error')
ax.plot(range(5), cap_df['Validation_error'], marker='s', label='Validation error')
ax.scatter([best_idx], [cap_df.iloc[best_idx]['Validation_error']], marker='*', s=170, label=f"Optimal tested capacity: {cap_df.iloc[best_idx]['Capacity']}")
ax.vlines(best_idx, cap_df.iloc[best_idx]['Training_error'], cap_df.iloc[best_idx]['Validation_error'], linestyles='dashed', label=f"Generalisation gap = {cap_df.iloc[best_idx]['Generalisation_gap']:.4f}")
ax.set_xticks(range(5), [f"{c}\n(depth={d})" for c,d in zip(cap_df['Capacity'],cap_df['Depth'])])
ax.set_xlabel('Model capacity'); ax.set_ylabel('Classification error rate'); ax.set_title('Training and Validation Error vs Model Capacity (C1-C5)')
ax.grid(True, alpha=.3); ax.legend(fontsize=8); fig.tight_layout(); fig.savefig(OUT/'capacity_training_validation_error.png', dpi=200); plt.show()

# %% [markdown]
# TASK 7 - BOOTSTRAP BIAS / VARIANCE AND MLE / MAP

# %%
Z_train = Z_train100; Z_val = Z_val100
n_classes = len(tomato_class_names); Y_val = np.eye(n_classes)[y_val]
```

```
rng = np.random.default_rng(42); B = 10; bv_rows = []
for name, depth in [('Low capacity (depth 2)', 2), ('High capacity (depth 32)', 32)]:
    prediction_replicas = []
    for b in range(B):
        idx = rng.integers(0, len(Z_train), size=len(Z_train))
        model = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=2, random_state=1000+b)
        model.fit(Z_train[idx], y_train[idx]); raw = model.predict_proba(Z_val)
        aligned = np.zeros((len(Z_val), n_classes))
        for col, cls in enumerate(model.classes_): aligned[:, int(cls)] = raw[:, col]
        prediction_replicas.append(aligned)
    P = np.stack(prediction_replicas)
    meanP = P.mean(axis=0)
    bias2 = np.mean(np.sum((meanP-Y_val)**2, axis=1))
    variance = np.mean(np.sum(np.var(P, axis=0, ddof=0), axis=1))
    brier = np.mean(np.sum((P-Y_val[None,:,:])**2, axis=2))
    bv_rows.append({'Model':name, 'Replicas':B, 'Bias_squared_proxy':bias2, 'Variance':variance, 'Mean_Brier_loss':brier, 'Residual':brier-bias2-variance})
print(pd.DataFrame(bv_rows).round(4).to_string(index=False))

```

```
# Beta(2,2) prior for a class proportion
with open(OUT/'tomato_class_names.json', encoding='utf-8') as f: names = json.load(f)
target_id = names.index('Tomato___Tomato_Yellow_Leaf_Curl_Virus')
rows=[]; rng = np.random.default_rng(2026)
for n in sorted(set([100,500,1000,5000,len(y_train)])):
    subset = y_train if n == len(y_train) else y_train[rng.choice(len(y_train),size=n,replace=False)]
    s = int(np.sum(subset == target_id)); mle=s/n; mapv=(s+1)/(n+2)
    rows.append({'n':n,'s':s,'MLE':mle,'MAP_Beta_2_2':mapv,'Absolute_difference':abs(mle-mapv)})
print(pd.DataFrame(rows).round(4).to_string(index=False))

# %% [markdown]
# TASK 8 - SOFTMAX REGRESSION GRADIENTS AND OPTIMISERS

# %%
# Loss: J = -mean(log(p_true)) + (lambda/2)*||W||_F^2
# Gradients: dW = X.T @ (P-Y)/N + lambda*W; db = mean(P-Y).
def stable_softmax(logits):
    shifted = logits - logits.max(axis=1, keepdims=True)
    expz = np.exp(shifted)
    return expz / expz.sum(axis=1, keepdims=True)

def loss_values(X, y, W, b, reg=1e-4, batch_size=1024):
    total = np.zeros(W.shape[0], dtype=float)
    n_total = len(y)
    for start in range(0, n_total, batch_size):
        end = min(start + batch_size, n_total)
        xb, yb = X[start:end], y[start:end]
        for r in range(W.shape[0]):
            logits = xb @ W[r] + b[r]
            m = logits.max(axis=1, keepdims=True)
            lse = m[:, 0] + np.log(np.exp(logits-m).sum(axis=1))
            total[r] += np.sum(lse - logits[np.arange(len(yb)), yb])
    return total/n_total + 0.5*reg*np.sum(W*W, axis=(1,2))

# Standardise PCA coordinates using train-only statistics, as in the experiment.
scaler100 = StandardScaler()
Xs_train = scaler100.fit_transform(Z_train100).astype(np.float32)
Xs_val = scaler100.transform(Z_val100).astype(np.float32)
```

```
rates = np.array([0.001, 0.01, 0.1]); epochs = 8; batch_size = 128; reg = 1e-4
rng = np.random.default_rng(42)
optimizer_summary = []; optimizer_histories = {}; optimizer_times = []

for strategy in ['batch', 'sgd', 'mini_batch']:
    W = np.zeros((len(rates), Xs_train.shape[1], len(tomato_class_names)), dtype=np.float64)
    b = np.zeros((len(rates), len(tomato_class_names)), dtype=np.float64)
    hist = {'train': [], 'val': []}; t0 = perf_counter()
    for epoch in range(epochs):
        perm = rng.permutation(len(Xs_train))
        if strategy == 'batch': batches = [np.arange(len(Xs_train))]
        elif strategy == 'mini_batch': batches = [perm[i:i+batch_size] for i in range(0,len(perm),batch_size)]
        else: batches = [perm[i:i+1] for i in range(len(perm))]
        for idx in batches:
            xb, yb = Xs_train[idx], y_train[idx]; bn = len(yb)
            logits = np.einsum('nd,rdc->rnc', xb, W) + b[:,None,:]
            P = stable_softmax(logits.reshape(-1, logits.shape[-1])).reshape(logits.shape)
            E = P.copy()
            E[np.arange(len(rates))[:,None], np.arange(bn)[None,:], yb[None,:]] -= 1.0
            gradW = np.einsum('nd,rnc->rdc', xb, E)/bn + reg*W
            gradb = E.mean(axis=1)
            W -= rates[:,None,None]*gradW
            b -= rates[:,None]*gradb
            if not np.isfinite(W).all() or not np.isfinite(b).all():
                raise FloatingPointError(f'Non-finite parameters for {strategy}')
        hist['train'].append(loss_values(Xs_train, y_train, W, b, reg))
        hist['val'].append(loss_values(Xs_val, y_val, W, b, reg))
    elapsed = perf_counter()-t0
    optimizer_histories[strategy] = hist
    optimizer_times.append({'Strategy':strategy,'Epochs':epochs,'Runtime_seconds':elapsed,'Runtime_per_epoch_seconds':elapsed/epochs})
    for ri, eta in enumerate(rates):
        optimizer_summary.append({'Strategy':strategy,'Learning_rate':eta,
            'Final_train_loss':hist['train'][-1][ri], 'Final_validation_loss':hist['val'][-1][ri],
            'Final_generalisation_gap':hist['val'][-1][ri]-hist['train'][-1][ri]})
    np.save(OUT/f'softmax_{strategy}_W.npy',W); np.save(OUT/f'softmax_{strategy}_b.npy',b)

```

```
opt_df = pd.DataFrame(optimizer_summary); opt_df.to_csv(OUT/'softmax_final_loss_results.csv',index=False)
time_df = pd.DataFrame(optimizer_times); time_df.to_csv(OUT/'softmax_optimizer_timing.csv',index=False)
print(opt_df.round(4).to_string(index=False)); print(time_df.round(4).to_string(index=False))

for strategy, hist in optimizer_histories.items():
    fig, ax = plt.subplots(figsize=(8, 5))
    for ri, eta in enumerate(rates):
        ax.plot(range(1,epochs+1),[h[ri] for h in hist['train']],marker='o',label=f'Train eta={eta:.3f}')
        ax.plot(range(1,epochs+1),[h[ri] for h in hist['val']],linestyle='--',label=f'Validation eta={eta:.3f}')
    ax.set_title(f'Softmax Regression Loss Curves - {strategy}'); ax.set_xlabel('Epoch')
    ax.set_ylabel('Regularised cross-entropy loss (nats)'); ax.grid(True,alpha=.3); ax.legend(fontsize=8,ncol=2)
    fig.tight_layout(); fig.savefig(OUT/f'softmax_loss_{strategy}.png',dpi=180); plt.show()

# %% [markdown]
# TASK 9 - PERFORMANCE AT REQUIRED DATASET SCALES

# %%
from sklearn.metrics import classification_report

scales = sorted(set([1000, 5000, 10000, len(y_train)]))
model_names = ['SGD_log_loss', 'KNN_k5', 'DecisionTree']
scale_rows=[]; class_metric_rows=[]; best_confusion = None
for rep, (Xa_full, Xv) in {'Raw_4096':(X_train,X_val),'PCA_100':(Z_train100,Z_val100)}.items():
    for n in scales:
        if n == len(y_train): idx = np.arange(len(y_train))
        else:
            idx, _ = train_test_split(np.arange(len(y_train)),train_size=n,stratify=y_train,random_state=100+n)
        Xa, ya = Xa_full[idx], y_train[idx]
        for name in model_names:
            mdl=build_model(name); t0=perf_counter()
            with warnings.catch_warnings(record=True) as cw:
                warnings.simplefilter('always', ConvergenceWarning); mdl.fit(Xa,ya)
            fit_s=perf_counter()-t0
            t0=perf_counter(); pred=mdl.predict(Xv); inf_s=perf_counter()-t0
            acc=accuracy_score(y_val,pred)
            pr,rc,f1,_=precision_recall_fscore_support(y_val,pred,labels=np.arange(len(tomato_class_names)),average='macro',zero_division=0)
            counts,typ = (mdl.coef_.size+mdl.intercept_.size,'coefficients+intercepts') if hasattr(mdl,'coef_') else ((mdl.tree_.node_count,'tree nodes') if
hasattr(mdl,'tree_') else (mdl._fit_X.size+len(mdl._y),'stored features+labels'))
```

```
            model_path = OUT / f'task9_{name}_{rep}_n{n}.joblib'
            joblib.dump(mdl, model_path)
            artifact_mb = model_path.stat().st_size / 1_000_000

scale_rows.append({'Training_images':n,'Representation':rep,'Model':name,'Accuracy':acc,'Macro_precision':pr,'Macro_recall':rc,'Macro_F1':f1,'Training_time_seconds':fit_s,'Validation_inference_seconds':inf_s,'Complexity_count':counts,'Complexity_type':typ,'Serialized_model_MB':artifact_mb})
            pcr,pcrr,pcf,pcs=precision_recall_fscore_support(y_val,pred,labels=np.arange(len(tomato_class_names)),average=None,zero_division=0)
            for ci,cname in enumerate(tomato_class_names):

class_metric_rows.append({'Training_images':n,'Representation':rep,'Model':name,'Class':cname,'Precision':pcr[ci],'Recall':pcrr[ci],'F1':pcf[ci],'Support':pcs[ci]})
```

```
            cm = confusion_matrix(y_val, pred, labels=np.arange(len(tomato_class_names)))
            cm_df = pd.DataFrame(cm, index=tomato_class_names, columns=tomato_class_names)
            cm_df.index.name = 'Actual'
            cm_df.to_csv(OUT/f'confusion_{name}_{rep}_n{n}.csv')
            if best_confusion is None or f1 > best_confusion['Macro_F1']:
                best_confusion = {'Macro_F1': f1, 'Model': name, 'Representation': rep,
                                  'Training_images': n, 'matrix': cm.copy()}
            print(n,rep,name,'accuracy=',f'{acc:.4f}','macroF1=',f'{f1:.4f}','fit=',f'{fit_s:.4f}','predict=',f'{inf_s:.4f}')
            del mdl; gc.collect()
scale_df=pd.DataFrame(scale_rows); class_df=pd.DataFrame(class_metric_rows)
scale_df.to_csv(OUT/'task9_model_performance_all_scales.csv',index=False)
class_df.to_csv(OUT/'task9_per_class_metrics.csv',index=False)
print(scale_df.round(4).to_string(index=False))
print('Best validation Macro-F1 configuration:', {k:v for k,v in best_confusion.items() if k != 'matrix'})
best_cm = best_confusion['matrix']
best_cm_df = pd.DataFrame(best_cm, index=tomato_class_names, columns=tomato_class_names)
best_cm_df.index.name = 'Actual'; best_cm_df.to_csv(OUT/'task9_best_validation_confusion.csv')
fig, ax = plt.subplots(figsize=(9,7)); im=ax.imshow(best_cm,aspect='auto'); fig.colorbar(im,ax=ax,label='Image count')
short=['Bacterial\nspot','Early\nblight','Late\nblight','Leaf\nMold','Septoria','Spider\nmites','Target\nSpot','Yellow Curl\nVirus','Mosaic\nvirus','Healthy']
ax.set_xticks(range(len(short)),short,fontsize=8); ax.set_yticks(range(len(short)),short,fontsize=8)
ax.set_xlabel('Predicted class'); ax.set_ylabel('Actual class'); ax.set_title('Validation Confusion Matrix - Best Macro-F1 configuration')
for ii in range(best_cm.shape[0]):
    for jj in range(best_cm.shape[1]): ax.text(jj,ii,str(best_cm[ii,jj]),ha='center',va='center',fontsize=7)
fig.tight_layout(); fig.savefig(OUT/'task9_confusion_matrix.png',dpi=180); plt.show()
# Most common class confusions excluding the diagonal.
conf_rows=[]
```

```
for ii in range(best_cm.shape[0]):
    for jj in range(best_cm.shape[1]):
        if ii != jj and best_cm[ii,jj] > 0:
            conf_rows.append({'Actual_class':tomato_class_names[ii], 'Predicted_class':tomato_class_names[jj], 'Misclassified_images':int(best_cm[ii,jj])})
conf_df=pd.DataFrame(conf_rows).sort_values('Misclassified_images',ascending=False)
print('Top 10 confusions:'); print(conf_df.head(10).to_string(index=False))

for metric,ylabel,title,file in [('Accuracy','Validation accuracy','Validation Accuracy vs Training Dataset
Size','task9_accuracy_vs_scale.png'),('Macro_F1','Validation macro-F1','Validation Macro-F1 vs Training Dataset Size','task9_macro_f1_vs_scale.png')]:
    fig,ax=plt.subplots(figsize=(9,5))
    for rep in ['Raw_4096','PCA_100']:
        for name in model_names:
            ss=scale_df[(scale_df.Representation==rep)&(scale_df.Model==name)].sort_values('Training_images')
            ax.plot(ss.Training_images,ss[metric],marker='o',label=f'{name} | {rep}')
    ax.set_xlabel('Training images'); ax.set_ylabel(ylabel); ax.set_title(title); ax.grid(True,alpha=.3); ax.legend(fontsize=7,ncol=2)
    fig.tight_layout(); fig.savefig(OUT/file,dpi=180); plt.show()

# %% [markdown]
# TASK 12 - 38-CLASS LABEL NOISE AND DESIGN CHALLENGE

# %%
from sklearn.preprocessing import StandardScaler

need38 = ['X38_train_uint8.npy','y38_train.npy','X38_val_uint8.npy','y38_val.npy','X38_test_uint8.npy','y38_test.npy','all38_class_names.json']
if not all((OUT/f).exists() for f in need38):
    raise FileNotFoundError('Run the 38-class preprocessing cell first.')
X38tr_u8=np.load(OUT/'X38_train_uint8.npy'); X38v_u8=np.load(OUT/'X38_val_uint8.npy'); X38t_u8=np.load(OUT/'X38_test_uint8.npy')
y38tr=np.load(OUT/'y38_train.npy').astype(int); y38v=np.load(OUT/'y38_val.npy').astype(int); y38t=np.load(OUT/'y38_test.npy').astype(int)
with open(OUT/'all38_class_names.json',encoding='utf-8') as f: names38=json.load(f)
N38=len(names38)

# Fit PCA only on training data, then fit scaling stats only on transformed train.
pca38=PCA(n_components=100,svd_solver='randomized',random_state=42,iterated_power=3)
t0=perf_counter(); Xfit=X38tr_u8.astype(np.float32)/255.0; pca38.fit(Xfit); pca_fit=perf_counter()-t0

def xform38(U,k):
    components=pca38.components_[:k].astype(np.float32); mean=pca38.mean_.astype(np.float32)
    output=np.empty((len(U),k),dtype=np.float32)
```

```
    for st in range(0,len(U),512):
        en=min(st+512,len(U)); xb=U[st:en].astype(np.float32)/255.0
        output[st:en]=(xb-mean)@components.T
    return output

Z38tr100=xform38(X38tr_u8,100); Z38v100=xform38(X38v_u8,100); Z38t100=xform38(X38t_u8,100)
scaler38=StandardScaler(); Z38tr100=scaler38.fit_transform(Z38tr100).astype(np.float32); Z38v100=scaler38.transform(Z38v100).astype(np.float32); Z38t100=scaler38.transform(Z38t100).astype(np.float32)
print('38-class PCA fit seconds:',f'{pca_fit:.4f}')
print('PCA100 cumulative explained variance:',f'{100*pca38.explained_variance_ratio_.sum():.4f}%')

# Exactly about 20% of training labels are changed. Validation/test are untouched.
rng=np.random.default_rng(42); noisy38=y38tr.copy(); m=int(round(.20*len(noisy38))); changed=rng.choice(len(noisy38),size=m,replace=False); offsets=rng.integers(1,N38,size=m); noisy38[changed]=(noisy38[changed]+offsets)%N38
noise_rate=np.mean(noisy38!=y38tr)
print('Corrupted labels:',int(np.sum(noisy38!=y38tr)),'/',len(noisy38),'noise rate:',f'{100*noise_rate:.4f}%')
np.save(OUT/'all38_train_labels_noisy20.npy',noisy38)

configs=[
('SGD_log_loss_clean_PCA100','log_loss',100,'clean'),
('SGD_log_loss_noisy20_PCA100','log_loss',100,'noisy'),
('SGD_modified_huber_noisy20_PCA100','modified_huber',100,'noisy'),
('SGD_modified_huber_noisy20_PCA64','modified_huber',64,'noisy'),
('KNN_noisy20_PCA100','knn',100,'noisy')]

def make38(kind):
    if kind=='log_loss': return SGDClassifier(loss='log_loss',alpha=1e-4,max_iter=100,tol=1e-3,random_state=42)
    if kind=='modified_huber': return SGDClassifier(loss='modified_huber',alpha=1e-4,max_iter=100,tol=1e-3,random_state=42)
    if kind=='knn': return KNeighborsClassifier(n_neighbors=5,weights='distance',algorithm='brute',n_jobs=-1)
    raise ValueError(kind)

def one_image_predict(raw_u8, k, model):
    flat=raw_u8.astype(np.float32).reshape(1,-1)/255.0
    transformed=(flat-pca38.mean_.astype(np.float32))@pca38.components_[:k].astype(np.float32).T
    scaled=(transformed-scaler38.mean_[:k].astype(np.float32))/scaler38.scale_[:k].astype(np.float32)
    return model.predict(scaled)[0]
```

```
0574
0575  lat_rng=np.random.default_rng(2026); lat_idx=lat_rng.choice(len(X38v_u8),size=min(100,len(X38v_u8)),replace=False)
0576  challenge_rows=[]; saved_models={}
0577  for name,kind,k,label_source in configs:
0578      Xtr=Z38tr100[:,:k]; Xv=Z38v100[:,:k]; Xt=Z38t100[:,:k]
0579      yfit=y38tr if label_source=='clean' else noisy38
0580      model=make38(kind); t0=perf_counter()
0581      with warnings.catch_warnings(record=True) as caught:
0582          warnings.simplefilter('always',ConvergenceWarning); model.fit(Xtr,yfit)
0583      fit_s=perf_counter()-t0
0584      warn='; '.join(str(w.message) for w in caught if issubclass(w.category,ConvergenceWarning))
0585      t0=perf_counter(); val_pred=model.predict(Xv); batch_predict=perf_counter()-t0
0586      acc=accuracy_score(y38v,val_pred); pr,rc,f1,_=precision_recall_fscore_support(y38v,val_pred,labels=np.arange(N38),average='macro',zero_division=0)
0587
artifact={'pca_mean':pca38.mean_.astype(np.float32),'pca_components':pca38.components_[:k].astype(np.float32),'scaler_mean':scaler38.mean_[:k].astype(np.flo
at32),'scaler_scale':scaler38.scale_[:k].astype(np.float32),'model':model,'n_components':k,'classes':names38}
0588      artifact_path=OUT/f'{name}.joblib'; joblib.dump(artifact,artifact_path,compress=3); size_mb=artifact_path.stat().st_size/1_000_000
0589      for ii in lat_idx[:5]: one_image_predict(X38v_u8[ii],k,model)
0590      lats=[]
0591      for ii in lat_idx:
0592          t0=perf_counter_ns(); one_image_predict(X38v_u8[ii],k,model); lats.append((perf_counter_ns()-t0)/1_000_000)
0593      p50=float(np.percentile(lats,50)); p95=float(np.percentile(lats,95))
0594
challenge_rows.append({'Configuration':name,'Training_labels':label_source,'Loss_or_model':kind,'PCA_components':k,'Validation_accuracy':acc,'Validation_m
acro_precision':pr,'Validation_macro_recall':rc,'Validation_macro_F1':f1,'Classifier_fit_seconds':fit_s,'Validation_batch_prediction_seconds':batch_predict,'Pipeli
ne_latency_mean_ms':float(np.mean(lats)),'Pipeline_latency_p50_ms':p50,'Pipeline_latency_p95_ms':p95,'Saved_pipeline_MB_decimal':size_mb,'Meets_5MB_a
rtifact_limit':size_mb<=5.0,'Meets_50ms_Kaggle_p95_proxy':p95<=50.0,'Convergence_warning':warn})
0595      saved_models[name]={'model':model,'artifact':artifact,'path':artifact_path,'k':k}
0596      print(name,'accuracy=',f'{acc:.4f}','MacroF1=',f'{f1:.4f}','fit=',f'{fit_s:.4f}','sizeMB=',f'{size_mb:.4f}','p50ms=',f'{p50:.4f}','p95ms=',f'{p95:.4f}')
0597      if warn: print('Convergence warning:',warn)
0598      del model,artifact; gc.collect()
0599
0600  challenge_df=pd.DataFrame(challenge_rows); challenge_df.to_csv(OUT/'task12_38class_adaptation_results.csv',index=False)
0601
print(challenge_df[['Configuration','Validation_accuracy','Validation_macro_F1','Pipeline_latency_p50_ms','Pipeline_latency_p95_ms','Saved_pipeline_MB_deci
mal','Meets_5MB_artifact_limit','Meets_50ms_Kaggle_p95_proxy']].round(4).to_string(index=False))
0602
0603  # Select the best noisy-label setting that meets the measured artifact and latency proxies.
```

```
eligible=challenge_df[(challenge_df.Training_labels=='noisy')&challenge_df.Meets_5MB_artifact_limit&challenge_df.Meets_50ms_Kaggle_p95_proxy]
if eligible.empty:
    print('No noisy-label configuration passed both measured proxy limits.')
else:
    chosen=eligible.sort_values('Validation_macro_F1',ascending=False).iloc[0]; name=chosen['Configuration']; mdl=saved_models[name]['model']; k=saved_models[name]['k']
    test_pred=mdl.predict(Z38t100[:,:k]); test_acc=accuracy_score(y38t,test_pred); _,_,test_f1,_=precision_recall_fscore_support(y38t,test_pred,labels=np.arange(N38),average='macro',zero_division=0)
    print('Selected:',name,'test_accuracy=',f'{test_acc:.4f}','test_macroF1=',f'{test_f1:.4f}')

# Entropy and majority-class baseline for train/validation.
```

```
for split, labels in [('Training',y38tr),('Validation',y38v)]:
    counts=np.bincount(labels,minlength=N38).astype(float); p=counts/counts.sum(); mask=p>0
    H=-np.sum(p[mask]*np.log2(p[mask])); Hmax=np.log2(N38); majority=int(np.argmax(counts))
    print(split,'entropy_bits=',f'{H:.4f}','max_entropy_bits=',f'{Hmax:.4f}','normalised_entropy=',f'{H/Hmax:.4f}','majority_class=',names38[majority],'majority_accuracy=',f'{counts[majority]/counts.sum():.4f}')

# End of consolidated reproducibility code.
```